



دانشگاه صنعتی ارومیه

دانشکده فناوری های صنعتی

گروه مهندسی کامپیوتر

عنوان :

تشخیص احساسات کامنت های شبکه های اجتماعی با استفاده از یادگیری عمیق

پژوهشگر :

زهراب عبدالرحیمی

استاد راهنما :

دکتر امیدرضا بلوکی اسپیلی

پروژه کارشناسی رشته مهندسی کامپیوتر گرایش هوش مصنوعی

بهار 1404

بِسْمِ اللَّهِ الرَّحْمَنِ الرَّحِيمِ

عنوان :

تشخیص احساسات کامنت های کاربران شبکه های اجتماعی با استفاده از یادگیری عمیق

چکیده

با گسترش روزافزون استفاده از شبکه‌های اجتماعی، حجم عظیمی از داده‌های متنی تولید می‌شود که می‌تواند حاوی اطلاعات ارزشمندی درباره نگرش‌ها و احساسات کاربران باشد. تحلیل احساسات یکی از شاخه‌های پردازش زبان طبیعی است که با هدف شناسایی و طبقه‌بندی احساسات موجود در متون انجام می‌شود.

یادگیری عمیق به عنوان یک زیرشاخه از هوش مصنوعی که توانایی تفسیر داده‌های پیچیده را دارد، این امکان را فراهم می‌کند تا احساسات کامنت‌های شبکه‌های اجتماعی به دقت شناسایی شوند که برای اهداف مارکتینگ و ارزیابی عملکرد به کار بروند.

در این پروژه، به طراحی و پیاده‌سازی یک مدل یادگیری عمیق برای تشخیص احساسات کامنت‌های کاربران در شبکه‌های اجتماعی پرداخته‌ایم. داده‌های مورد استفاده شامل نظرات کاربران با برچسب‌های احساسی مثبت، منفی و خنثی بوده و مراحل پیش‌پردازش، استخراج ویژگی‌ها، آموزش و سپس تست مدل‌ها به طور کامل انجام شده است. هدف اصلی این پروژه، تشخیص احساسات و فراهم کردن بستری برای تحلیل نگرش کاربران نسبت به موضوعات مختلف است. نتایج به دست آمده نشان می‌دهد که مدل پیشنهادی عملکرد قابل قبولی در طبقه‌بندی احساسات دارد و می‌تواند به عنوان ابزاری مؤثر در تحلیل نظرات آنلاین مورد استفاده قرار گیرد.

کلمات کلیدی: هوش مصنوعی - یادگیری عمیق - مارکتینگ - پردازش زبان طبیعی - مدل‌های از پیش آموزش دیده -

فهرست مطالب

9.....	فصل اول مقدمه.....	1.
10.....	۱-۱ تاریخچه مسئله.....	
11.....	۲-۱ معیار های ارزیابی عملکرد مدل های هوش مصنوعی.....	
14.....	۳-۱ سرفصل ها.....	
15.....	فصل دوم مروری بر هوش مصنوعی.....	2.
16.....	۱-۲ انواع هوش مصنوعی.....	
17.....	۲-۲ انواع مدل های یادگیری عمیق.....	
20.....	فصل سوم کدنویسی و اجرای پروژه.....	3.
22.....	۱-۳ بررسی کتابخانه ها.....	
24.....	۲-۳ بررسی و مصورسازی دیتاست ها.....	
49.....	۳-۳ پیش پردازش.....	
59.....	فصل چهارم نتایج.....	۴.
67.....	فهرست مراجع و منابع.....	5.

فهرست تصاویر

22.....	تصویر 3-1 کتابخانه‌های مورد استفاده
24.....	تصویر 3-2 کد بدست آوردن دایرکتوری دیتاست‌ها
25.....	تصویر 3-3 خروجی قطعه کد قبل
26.....	تصویر 3-4 تکه کد نمایش کلی کردن دیتاست اول
26.....	تصویر 3-5 نتیجه قطعه کد قبل
26.....	تصویر 3-6 تکه کد نمودار میله‌ای دیتاست آموزش دیتاست اول
27.....	تصویر 3-7 نمودار sentimental distribution مربوط به دیتاست آموزش دیتاست اول
28.....	تصویر 3-8 کد داده‌های ناقص و جاافتاده مربوط به دیتاست آموزش دیتاست اول
28.....	تصویر 3-9 خروجی کد داده‌های ناقص
29.....	تصویر 3-10 کد نمودار هیستوگرام مربوط به دیتاست آموزش دیتاست اول
30.....	تصویر 3-11 نمودار هیستوگرام طول کامنت‌های مربوط به دیتاست آموزش دیتاست اول
30.....	تصویر 3-12 کد نمودار همبستگی مربوط به دیتاست آموزش دیتاست اول
32.....	تصویر 3-13 نمودار همبستگی مربوط به دیتاست آموزش دیتاست اول
32.....	تصویر 3-14 box-plot مربوط به دیتاست آموزش دیتاست اول
33.....	تصویر 3-15 نمودار box-plot مربوط به دیتاست آموزش دیتاست اول
34.....	تصویر 3-16 کد نمایش کلی دیتاست تست دیتاست اول
35.....	تصویر 3-17 دیتاست تست دیتاست اول
35.....	تصویر 3-18 کد نمودار میله‌ای دیتاست تست دیتاست اول
36.....	تصویر 3-19 نمودار میله‌ای دیتاست تست دیتاست اول
36.....	تصویر 3-20 کد تشخیص و حذف داده‌های ناقص دیتاست تست دیتاست اول
37.....	تصویر 3-21 نتیجه قطعه کد قبل
38.....	تصویر 3-22 کد نمودار طول کامنت‌های دیتاست تست دیتاست اول
39.....	تصویر 3-23 نمودار هیستوگرام دیتاست تست دیتاست اول
39.....	تصویر 3-24 کد نمودار همبستگی دیتاست تست دیتاست اول
40.....	تصویر 3-25 نمودار همبستگی مربوط به دیتاست تست دیتاست اول
41.....	تصویر 3-26 قطعه کد نمودار box-plot دیتاست تست دیتاست اول
42.....	تصویر 3-27 نمودار box-plot مربوط به دیتاست تست دیتاست اول

42.....	تصویر 3-28 کد نمایش اولیه دیتاست دوم
43.....	تصویر 3-29 دیتاست دوم
43.....	تصویر 3-30 کد نمودار میله‌ای مربوط به دیتاست دوم
44.....	تصویر 3-31 نمودار میله‌ای مربوط به دیتاست دوم
45.....	تصویر 3-32 کد نمودار هیستوگرام مربوط به دیتاست دوم
46.....	تصویر 3-33 نمودار هیستوگرام مربوط به دیتاست دوم
46.....	تصویر 3-34 کد نمودار box-plot مربوط به دیتاست دوم
47.....	تصویر 3-35 نمودار box-plot مربوط به دیتاست دوم پیش از پیش‌پردازش
48.....	تصویر 3-36 کد نمودار همبستگی مربوط به دیتاست دوم
49.....	تصویر 3-37 نمودار همبستگی مربوط به دیتاست دوم
49.....	تصویر 3-38 کد متعادل‌سازی مربوط به دیتاست دوم
50.....	تصویر 3-39 کد مربوط به ترکیب کردن دیتاست‌ها و حذف ویژگی‌های نامرتب
51.....	تصویر 3-40 کد مربوط به توکنایزر
52.....	تصویر 3-41 کد تنظیم وزن کلاس‌ها و تقسیم دیتاست به دو بخش آموزش و تست
53.....	تصویر 3-42 کد بارگذاری بردارهای آموزش دیده GloVe
54.....	تصویر 3-43 کد مربوط به ماتریس embedding مدل
55.....	تصویر 3-44 کد مربوط به مدل BILSTM
56.....	تصویر 3-45 شرح عملکرد لایه‌های BILSTM
57.....	تصویر 3-46 کد مربوط به کامپایل مدل و تنظیم callbacks
57.....	تصویر 3-47 کد مربوط به فرایند آموزش مدل
60.....	تصویر 3-48 کد مربوط به تست مدل
61.....	تصویر 3-49 بخش اول نتایج مدل
62.....	تصویر 3-50 بخش دوم نتایج مدل
64.....	تصویر 3-51 کد مربوط به ترسیم نمودارهای آموزش و تست مدل
65.....	تصویر 3-52 کد بررسی داده‌های تست و آموزش استفاده شده در فرایند آموزش مدل
66.....	تصویر 3-53 نمودارهای نتایج عملکرد مدل

1. فصل اول

مقدمه

مسئله‌ای که این پروژه به آن می‌پردازد، تشخیص خودکار احساسات در کامنت‌های کاربران شبکه‌های اجتماعی است. منظور از تشخیص احساسات، شناسایی نگرش عاطفی نویسنده نسبت به موضوع مورد بحث است، که در این پروژه در یکی از سه دسته «مثبت»، «منفی» یا «خنثی» قرار می‌گیرد. این کار نه تنها به درک بهتر مخاطبان کمک می‌کند، بلکه می‌تواند در تصمیم‌گیری‌های کلان در حوزه‌های بازاریابی، سیاست، فرهنگ و مدیریت خدمات نیز مؤثر واقع شود.

۱-۱ تاریخچه مسئله

تاریخچه تحلیل احساسات به سال‌های ابتدایی دهه ۲۰۰۰ بازمی‌گردد، زمانی که محققان به بررسی نگرش افراد در بازخوردهای آنلاین پرداختند. در ابتدا، این حوزه بیشتر بر تحلیل نظرات کاربران در مورد محصولات تمرکز داشت.

امروزه با پیشرفت‌های حوزه یادگیری ماشین و پردازش زبان طبیعی (NLP)، ابزارهای تحلیل احساسات دقیق‌تر و گسترده‌تری توسعه یافتند. به‌ویژه در دهه گذشته، با ظهور مدل‌های پیشرفته‌تری همچون BERT و LSTM، امکان تحلیل دقیق‌تر از نظر معنایی احساسات در متون فراهم شده است.

کاربردهای تحلیل احساسات بسیار گسترده است. در حوزه بازاریابی دیجیتال، شرکت‌ها می‌توانند از تحلیل کامنت‌ها برای ارزیابی بازخورد مشتریان نسبت به محصولات و خدمات خود بهره ببرند. در سیاست و رسانه، تحلیل احساسات می‌تواند در پایش افکار عمومی، تشخیص بحران‌های احتمالی و طراحی کمپین‌های تبلیغاتی مؤثر باشد. به‌ویژه در انتخابات نقش مهمی را ایفا کند. همچنین در مدیریت شبکه‌های اجتماعی، ابزارهای تشخیص احساسات می‌توانند به‌طور خودکار نظرات منفی یا توهین‌آمیز را شناسایی کرده و در فیلتر کردن محتوای نامناسب مؤثر باشند.

این پروژه تلاش دارد با استفاده از یادگیری عمیق، مدلی کارآمد برای طبقه‌بندی احساسات کامنت‌های کاربران طراحی و پیاده‌سازی کند. مراحل پروژه شامل انتخاب دیتاست مناسب، پیش‌پردازش دیتاست، آموزش مدل و ارزیابی دقت آن خواهد بود. هدف نهایی، آموزش مدل هوش مصنوعی با استفاده از یادگیری عمیق است که بتواند با دقت بالا احساسات کاربران را تشخیص دهد و به عنوان ابزاری هوشمند در تحلیل داده‌های شبکه‌های اجتماعی مورد استفاده قرار گیرد.

۱-۲ معیارهای ارزیابی عملکرد مدل های هوش مصنوعی

در ارزیابی مدل های یادگیری ماشین به ویژه در مسائل طبقه بندی مانند تشخیص احساسات، استفاده از معیارهای دقیق و استاندارد برای سنجش عملکرد مدل ضروری است. هدف از این ارزیابی ها، سنجش دقت، کارایی و توانایی مدل در شناسایی درست برچسب های احساسی (مثبت، منفی، خنثی) در داده های جدید و ناشناخته است. انتخاب معیار مناسب بستگی به نوع داده، هدف پروژه و حساسیت نسبت به خطاها دارد.

در ادامه مهم ترین معیارهای ارزیابی عملکرد در این حوزه معرفی و شرح داده شده اند.

۱. دقت (Accuracy)

دقت یکی از ساده ترین و پرکاربردترین معیارهای ارزیابی در مسائل طبقه بندی است. این معیار نشان می دهد که مدل در مجموع چه درصدی از نمونه ها را به درستی طبقه بندی کرده است. این معیار در حالتی مناسب است که تعداد نمونه های هر کلاس تقریباً برابر باشد. اما در شرایطی که داده نامتوازن باشد (مثلاً کلاس خنثی بسیار بیشتر از کلاس منفی است)، accuracy می تواند گمراه کننده باشد، چرا که ممکن است مدل با پیش بینی صرفاً کلاس غالب به دقت بالا برسد که گاهی هدف از پروژه تشخیص کلاس های نادرتر می باشد. فرمول محاسبه:

$$\frac{TP + TN}{TP + TN + FP + FN} = Accuracy$$

که در آن:

- TP (True Positive): تعداد نمونه های مثبت که درست تشخیص داده شده اند.
- TN (True Negative): تعداد نمونه های منفی که درست تشخیص داده شده اند.
- FP (False Positive): تعداد نمونه های منفی که اشتباهاً مثبت تشخیص داده شده اند.
- FN (False Negative): تعداد نمونه های مثبتی که اشتباهاً منفی تشخیص داده شده اند.

۲. دقت کلاس مثبت (Precision)

precision نشان می‌دهد که از میان تمام نمونه‌هایی که مدل به عنوان "مثبت" پیش‌بینی کرده، چند درصد واقعاً مثبت بوده‌اند. این معیار حساس به اشتباهات مثبت کاذب (False Positives) است. اگر precision بالا باشد، به این معناست که مدل در هنگام پیش‌بینی نمونه‌های مثبت کمتر اشتباه می‌کند. این معیار در مسائلی مانند فیلتر کردن محتوای توهین‌آمیز یا تشخیص نظرات مثبت برای استفاده در بازاریابی، precision اهمیت بالایی دارد. زیرا نمی‌خواهیم محتوای منفی یا خنثی به اشتباه به عنوان مثبت در نظر گرفته شود.

فرمول:

$$\frac{TP}{TP + FP} = precision$$

۳. بازیابی (Recall) یا حساسیت

Recall معیار است که نشان می‌دهد که از میان همه نمونه‌هایی که واقعاً مثبت هستند، مدل چه تعداد را به درستی تشخیص داده است. این معیار حساس به اشتباهات منفی کاذب (False Negatives) است. اگر recall بالا باشد، مدل توانایی خوبی در شناسایی تمام نمونه‌های مثبت دارد. در مسائل حساس مانند تشخیص نظرات منفی در بحران‌های روابط عمومی یا شناسایی نارضایتی مشتریان، recall اهمیت بالایی دارد. زیرا نمی‌خواهیم موارد منفی از چشم مدل پنهان بمانند.

فرمول:

$$\frac{TP}{TP + FN} = Recall$$

۴. F1-measure (F1 Score)

F1-measure یک امتیازی ترکیبی است که میانگین هارمونیک بین precision و recall را محاسبه می‌کند. این معیار زمانی اهمیت دارد که نیاز به تعادل بین precision و recall وجود داشته باشد. F1 در مواقعی که یک معیار به تنهایی نمی‌تواند عملکرد مدل را به خوبی توصیف کند، مفید است. این معیار به ویژه در داده‌های

نامتوازن که کلاس‌های مثبت و منفی تعداد برابری ندارند کاربرد زیادی دارد. در اکثر پروژه‌های عملی از جمله تحلیل احساسات شبکه‌های اجتماعی، F1 score معیار مناسبی برای مقایسه بین مدل‌ها محسوب می‌شود.

فرمول:

$$\frac{precision \times recall}{precision + recall} \times 2 = f1_measure$$

۵. ماتریس آشفتگی (Confusion Matrix)

این معیار ماتریسی است که توزیع دقیق پیش‌بینی‌های مدل در مقابل مقادیر واقعی را نشان می‌دهد. ماتریس آشفتگی دیدی جامع از عملکرد مدل فراهم می‌کند و نشان می‌دهد که مدل در چه کلاس‌هایی بیشترین خطا را دارد. برای تحلیل دقیق عملکرد مدل در هر کلاس بسیار مفید است.

شکل کلی این ماتریس (برای حالت سه کلاس):

پیش‌بینی \ واقعی	مثبت positive	خنثی neutral	منفی negative
مثبت positive	TP	FN1	FN2
خنثی neutral	FP1	TN	FN3
منفی negative	FP2	FP3	TN

جمع‌بندی و انتخاب معیار مناسب

انتخاب معیار مناسب برای ارزیابی مدل باید بر اساس ویژگی‌های داده‌ها، هدف پروژه، حساسیت به نوع خطاها و کاربرد نهایی مدل انجام شود. در پروژه‌هایی مانند تشخیص احساسات کامنت‌های کاربران شبکه‌های اجتماعی، معمولاً از ترکیبی از دقت کلی، F1 Score، و ماتریس آشفتگی استفاده می‌شود تا ارزیابی دقیقی از عملکرد مدل به‌دست آید.

برای مثال:

- اگر هدف شناسایی دقیق نظرات منفی باشد (مثلاً برای جلوگیری از بحران یا فیلتر محتوا)، باید بیشتر بر recall کلاس منفی تمرکز کرد.
- اگر هدف استفاده از نظرات مثبت در بازاریابی باشد، precision کلاس مثبت اهمیت بیشتری خواهد داشت.

۱-۳ سرفصل‌ها

در فصل دوم گزارش این پروژه، به برخی از مفاهیم اولیه هوش مصنوعی خواهیم پرداخت از دیپ لرنینگ کمی صحبت می‌کنیم و مدل‌های مناسب و قابل استفاده برای این پروژه همچنین نحوه عملکرد آنها صحبت می‌کنیم.

در فصل سوم به بخش کدهای پروژه می‌رسیم. کتابخانه‌ها و ماژول‌های استفاده شده را بررسی می‌کنیم. بخش بخش کد شامل نحوه بررسی و نمایش دیتاست، نرمالایز کردن و استفاده از داده‌های موجود، ساختن مدل، اجرای مدل، تست کردن مدل و رسم نمودارهای لازم، و همچنین یک تکه کد که شبیه سازی آزمایشگاه را برای کاربر فراهم می‌کند، را خواهیم داشت. در تمام پروژه از زبان پایتون استفاده شده است.

فصل چهارم شامل ارزیابی نتایج مدل ساخته شده و اینکه تا چه حد مدل با توجه به داده‌های ما، به واقعیت نزدیک است

2. فصل دوم

مروری بر هوش مصنوعی

۲-۱ انواع هوش مصنوعی

تا به اینجا به مروری از تاریخچه هوش مصنوعی پرداختیم و اینکه چرا این مبحث دارای اهمیت فراوانی است. حال به تعریف مفاهیمی از هوش مصنوعی مرتبط با این پروژه می‌پردازیم.

شبکه‌های عصبی (Neural Networks): در لایه‌هایی سازماندهی می‌شوند که با خطوطی شبیه‌سازی شده به یکدیگر متصل هستند. بالاترین لایه، لایه‌ی دریافت است. این لایه مانند حسگری عمل می‌کند که اطلاعات را برای پردازش دریافت می‌کند و آن‌ها را به لایه‌های پایین‌تر می‌فرستد. این فرآیند پس از دریافت اطلاعات با حداقل دو لایه‌ی دیگر ادامه پیدا می‌کند که به صورت سلسه‌مراتبی روی هم قرار دارند و اطلاعات را با استفاده از پیوندها دسته‌بندی و ارسال می‌کنند. در پایین‌ترین بخش سلسله‌مراتب لایه‌ی خروجی قرار دارد که به طور معمول تعداد نورون‌های مصنوعی آن از تمام لایه‌های دیگر کمتر است. این لایه داده‌های محاسبه شده را به فرمتی تبدیل می‌کند که برای ماشین قابل خواندن باشد. اساس کار این روش یادگیری ماشین (Machine Learning) است. سیستمی است که تجربه را به دانش تبدیل می‌کند. این پروسه به سیستم این توانایی را می‌دهد که الگوها و قوانین را با سرعتی که همواره در حال افزایش است شناسایی کند. در انواع مختلف یادگیری ماشینی از هر دو نوع هوش مصنوعی نمادین و عصبی استفاده می‌شود.

پردازش زبان طبیعی (Natural Language Processing) NLP: شاخه‌ای از علوم کامپیوتر و هوش مصنوعی است که به تعامل بین رایانه و زبان انسانی می‌پردازد. هدف اصلی NLP، توانمندسازی رایانه‌ها برای درک، تفسیر، تولید و تحلیل زبان طبیعی انسان‌هاست. از جمله کاربردهای مهم NLP میتوان به موارد زیر اشاره کرد مثل ترجمه ماشینی (Google Translate)، دستیارهای هوشمند (Google Assistant)، چت‌بات‌ها و تشخیص احساسات کاربران از متون و نظرات (Sentimental Analysis) که در این پروژه، تمرکز ما بر تحلیل احساسات در متن‌های کاربران شبکه‌های اجتماعی است، که مدل باید بتواند از روی کلمات و ساختار جمله، نوع احساس (مثبت، منفی یا خنثی) را به درستی تشخیص دهد.

یادگیری عمیق (Deep Learning): زیرگونه‌ای از یادگیری ماشینی است. در این مورد تنها از هوش مصنوعی عصبی یا همان شبکه‌های عصبی استفاده می‌شود. اکثر کاربردهای امروزی هوش مصنوعی بر پایه‌ی یادگیری عمیق هستند. به لطف امکان گسترش دادن سریع طراحی شبکه‌های عصبی و تبدیل کردن آن‌ها به

سیستم‌هایی پیچیده‌تر و قوی‌تر با لایه‌های جدید، مقیاس یادگیری عمیق را می‌توان به سادگی تغییر داد و آن را با کاربردهای بسیار زیادی منطبق کرد.

۲-۲ انواع مدل‌های یادگیری عمیق

در این بخش به انواع مدل‌های هوش مصنوعی و شرح مختصری از آنها می‌پردازیم که می‌توان با استفاده از آنها به اهداف این پروژه دست یافت و چنین پروژه‌ای را پیاده‌سازی کرد. سپس به روشی که از آن در این پروژه استفاده شده است با جزئیات بیشتری می‌پردازیم.

شبکه عصبی بازگشتی (Recurrent Neural Network): RNN: از جمله مدل‌های ابتدایی یادگیری عمیق برای پردازش داده‌های ترتیبی مانند متن است. این شبکه با داشتن حافظه‌ای که اطلاعات مراحل قبلی را به مراحل بعدی منتقل می‌کند، قادر است توالی واژه‌ها را درک کرده و وابستگی زمانی بین آن‌ها را تشخیص دهد. به عبارت دیگر، RNN هنگام تحلیل یک جمله، تأثیر کلمات قبلی را بر درک کلمات بعدی لحاظ می‌کند. با وجود کارایی بالا در جملات کوتاه، این مدل در مواجهه با وابستگی‌های بلندمدت در جملات دچار مشکلاتی مثل فراموشی اطلاعات اولیه، مواجهه می‌شود.

مدل GRU (Gated Recurrent Unit): مشابه LSTM برای پردازش توالی‌ها طراحی شده است، اما معماری ساده‌تری دارد. این مدل تنها از دو دروازه — دروازه بروزرسانی و دروازه بازنشانی — استفاده می‌کند و در مقایسه با LSTM به پارامترهای کمتری نیاز دارد. با وجود این سادگی، GRU عملکردی تقریباً مشابه LSTM دارد و به‌ویژه در تحلیل سریع‌تر و کم‌هزینه‌تر داده‌های متنی مفید است. در پروژه‌هایی که محدودیت منابع محاسباتی وجود دارد، GRU انتخاب مناسبی برای تحلیل احساسات خواهد بود.

شبکه عصبی کانولوشنال (Convolutional Neural Network): CNN: در حالی که شبکه‌های عصبی پیچشی یا CNN بیشتر در پردازش تصویر معروف هستند، از آن‌ها می‌توان برای تحلیل متن نیز استفاده کرد. در این کاربرد، CNN با استفاده از فیلترهای پیچشی به دنبال الگوهای محلی در بین کلمات است، مانند ترکیب‌های دو یا سه کلمه‌ای که بار احساسی دارند. این مدل به‌طور خاص برای تحلیل نظرات کوتاه مانند CNN توییت‌ها یا کامنت‌های شبکه‌های اجتماعی مناسب است. با سرعت بالا و کارایی خوب، می‌تواند

ویژگی‌های مهم احساسی را از متن استخراج کند، هر چند برای درک وابستگی‌های بلندمدت مانند LSTM و GRU ساخته نشده است.

مدل LSTM (Long Short-Term Memory): نسخه‌ای بهبود یافته از RNN است که برای رفع مشکل فراموشی اطلاعات بلندمدت طراحی شده است. با استفاده از سه نوع دروازه که دروازه ورودی، دروازه خروجی و دروازه فراموشی هستند، جریان اطلاعات را در طول توالی کنترل می‌کند و امکان ذخیره یا حذف اطلاعات خاصی از حافظه را دارد. این ویژگی باعث می‌شود LSTM بتواند وابستگی‌های معنایی طولانی در جملات را بهتر درک کند. در تحلیل احساسات، LSTM به‌طور خاص برای پردازش جملات طولانی یا متونی با ساختار پیچیده مؤثر است.

مدل BiLSTM (Bidirectional LSTM): نسخه‌ای پیشرفته از LSTM است که متن را از دو جهت یعنی هم از ابتدا به انتها و هم از انتها به ابتدا تحلیل می‌کند. این ساختار دوطرفه به مدل کمک می‌کند تا زمینه (Context) بهتری از هر واژه در جمله به دست آورد، چون می‌داند چه چیزی قبل و بعد از آن آمده است. در تحلیل احساسات، BiLSTM در درک ظرافت‌های زبانی و احساساتی پنهان بسیار مؤثر است، به‌ویژه در زبان‌هایی که ترتیب واژگان متغیر است یا معانی کلمات به جایگاهشان در جمله وابسته‌اند.

مدل BERT (Bidirectional Encoder Representations from Transformers): یکی از پیشرفته‌ترین مدل‌های NLP است که توسط گوگل معرفی شده است. این مدل مبتنی بر معماری ترنسفورمر بوده و توانایی درک هم‌زمان جمله از هر دو جهت را دارد. BERT برخلاف مدل‌های RNN یا LSTM، جملات را به‌صورت کامل (و نه واژه‌به‌واژه) پردازش می‌کند. قابلیت fine-tuning این مدل روی داده‌های خاص، آن را به گزینه‌ای عالی برای تحلیل دقیق احساسات کاربران در شبکه‌های اجتماعی تبدیل کرده است. دقت بالا، درک دقیق‌تر از معنا، و پشتیبانی از جملات پیچیده از ویژگی‌های بارز BERT است.

مدل RoBERTa (Robustly Optimized BERT Approach): نسخه‌ای بهینه‌سازی شده از مدل BERT است. این مدل با استفاده از همان معماری پایه‌ی BERT ساخته شده است اما تغییراتی کلیدی در فرآیند آموزش آن اعمال شده تا عملکرد بهتری در درک زبان طبیعی داشته باشد. از جمله این تغییرات می‌توان به

استفاده از حجم بسیار بیشتر داده‌های متنی، حذف پیش‌وظیفه‌ی (NSP (Next Sentence Prediction، استفاده از طول توالی‌های بلندتر، و تعداد بیشتری از تکرارهای آموزش اشاره کرد. هدف از طراحی RoBERTa افزایش دقت و قدرت تعمیم‌دهی مدل در وظایف مختلف پردازش زبان طبیعی از جمله تحلیل احساسات و دسته‌بندی متن است. RoBERTa نیز مانند BERT از معماری Transformer بهره می‌برد و توانایی پردازش جملات به صورت دوطرفه (Bidirectional) را دارد. در پروژه‌های تحلیل احساسات، RoBERTa اغلب دقت بالاتری نسبت به BERT ارائه می‌دهد، به‌خصوص زمانی که داده‌ها پیچیده یا چندمعنایی باشند. این مدل نیازمند منابع محاسباتی بالاتری است، اما نتایج حاصل از آن نیز پیشرفته‌تر و دقیق‌تر است.

3. فصل سوم

کدنویسی و اجرای پروژه

در این پروژه از مدل BiLSTM است که همانطور که گفته شد، مدل BiLSTM نوعی شبکه عصبی بازگشتی (RNN) پیشرفته است که برای پردازش دنباله‌ای از داده‌ها (مثل جملات) طراحی شده است. برخلاف مدل LSTM ساده که فقط اطلاعات را از ابتدا به انتهای دنباله پردازش می‌کند (مثلاً از کلمه اول جمله به کلمه آخر)، مدل BiLSTM دنباله را از هر دو جهت (از ابتدا به انتها و از انتها به ابتدا) می‌خواند. این کار باعث می‌شود که مدل هم اطلاعات قبلی و هم اطلاعات بعدی هر کلمه را داشته باشد و در نتیجه درک دقیق‌تری از معنای جمله یا عبارت به دست آورد. این ویژگی به‌ویژه در مسائل پردازش زبان طبیعی (NLP) مثل تحلیل احساسات، ترجمه ماشینی، یا تشخیص موجودیت‌ها بسیار مفید است. در عمل، BiLSTM از دو لایه LSTM استفاده می‌کند به این صورت که یکی به صورت معمول (forward) و یکی به صورت معکوس (backward) هر کلمه یا عنصر در دنباله همزمان به‌وسیله‌ی این دو لایه پردازش می‌شود و خروجی آن‌ها با هم ترکیب می‌گردد. مثلاً اگر جمله‌ای را بررسی می‌کنیم، مدل می‌تواند تشخیص دهد که کلمه‌ای مثل "بد" اگر بعد از "نه" آمده، به معنی مثبت است که این نوع درک فقط وقتی ممکن است که مدل بتواند هم به عقب و هم به جلو در جمله نگاه کند. به همین دلیل، BiLSTM در مقایسه با LSTM معمولی دقت بالاتری در تحلیل متن دارد، چون دید گسترده‌تری نسبت به کل ساختار جمله دارد.

همانطور که گفته شد، از زبان پایتون برای برنامه نویسی این پروژه استفاده شده است همچنین این پروژه در Kaggle نوشته شده است. در ادامه نه تنها کد پروژه را بررسی میکنیم بلکه به بررسی دیتاست‌ها، پیش‌پردازش آنها، آموزش مدل و تست آن میپردازیم که این تحلیل را خط به خط و همانگونه که پروژه نوشته شده است، انجام میدهیم. روبه‌رو و یا بالای هر خط کد که در عکس مربوط به هر قطعه کد که ان را شرح میدهیم و تحلیل میکنیم، یک توضیح مختصر در قالب کامنت نوشته‌ایم تا کدخوانانتر باشد.

```
import numpy as np # for numerical operations, especially for arrays, mathematical functions and etc
import pandas as pd # to read datasets, clean data, filter rows, select columns, handle missing values and etc
import matplotlib.pyplot as plt #for plotting charts, such as line plots, histograms, bar charts and etc
import seaborn as sns # informative plots like heatmaps, distribution plots, and box plots
from collections import Counter # to count how many times each word, class label, or token appears
import os # to interact with the file system
import tensorflow as tf # to build and train neural networks
from tensorflow.keras.preprocessing.text import Tokenizer # to Converts words into integer tokens
from tensorflow.keras.preprocessing.sequence import pad_sequences # to Makes all input sequences the same length by padding them with zeros
from tensorflow.keras.models import Sequential # to create a model layer-by-layer
# word indices into dense vectors, read the sequence forward and backward, Fully connected layer,
# Randomly deactivates some neurons during training to prevent overfitting
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau # Stops training if the model stops improving on the validation set
from tensorflow.keras.regularizers import L1, L2 # to apply L1 or L2 regularization to layers to Prevent overfitting by penalizing large weights
from tensorflow.keras.models import Model # to build more complex architectures
from tensorflow.keras.layers import Input # to define an input layer manually
from sklearn.feature_extraction.text import TfidfVectorizer # to turns text into numerical vectors using Term Frequency-Inverse Document Frequency
from sklearn.model_selection import train_test_split # to Split dataset into training and testing sets
from sklearn.preprocessing import LabelEncoder # to Converts categorical labels ('positive', 'negative') into integers (0, 1)
from sklearn.utils.class_weight import compute_class_weight # Computes weights for each class to balance them
from sklearn.metrics import classification_report, confusion_matrix # Shows precision, recall, F1-score
```

تصویر ۳-۱ کتابخانه‌های مورد استفاده

همانطور که در تصویر مشاهده میکنید از تعدادی کتابخانه استفاده شده است که کاربرد آنها به شرح زیر است:

1. از کتابخانه numpy برای دسترسی و استفاده از عملیات عددی و کار با آرایه‌ها استفاده کرده ایم.
2. از کتابخانه pandas برای بارگذاری داده‌ها، پاک‌سازی، فیلتر کردن ردیف‌ها و ستون‌ها، و بررسی داده‌های گمشده استفاده کرده ایم.
- این دو کتابخانه به عنوان پایه‌های کار با داده در پایتون و مدیریت داده‌ها مورد استفاده قرار گرفته اند.
3. از کتابخانه matplotlib.pyplot در مصورسازی داده‌ها و دیتاست‌ها برای رای رسم نمودارهای خطی، ستونی و هیستوگرام استفاده شده است.
4. از کتابخانه seaborn برای نمایش بصری داده‌ها با نمودارهایی مانند Heatmap، Boxplot و Distribution plot استفاده شده است.
5. از کتابخانه collections برای شمارش فراوانی کلمات یا برچسب‌های کلاس استفاده شده است.

به طور کلی از این سه کتابخانه برای بصری‌سازی دیتاست‌ها استفاده شده است تا الگوها و مشکلات احتمالی داده‌ها شناسایی، درک و در صورت نیاز حل شوند.

6. از کتابخانه os برای کار با فایل ها و مسیرهای سیستم فایل استفاده شده است.
7. از کتابخانه tensorflow برای طراحی و ساخت مدل های یادگیری عمیق استفاده شده است که البته از زیرمجموعه keras استفاده شده است که در ادامه انها را بررسی میکنیم.
8. از کتابخانه `tensorflow.keras.preprocessing.text.Tokenizer` از Keras نیز برای تبدیل کلمات به توکن های عددی استفاده شده است.
9. از کتابخانه `tensorflow.keras.preprocessing.sequence.pad_sequences` از pad_sequences برای هم طول کردن ورودی ها از طریق افزودن صفر در انتها انها استفاده شده است.
10. از کتابخانه `tensorflow.keras.models.Sequential` کلاس Sequential از `tensorflow.keras.models` امکان ساخت مدل به صورت لایه به لایه را فراهم می کند که از ما هم در این جهت استفاده کرده ایم.
11. از کتابخانه `tensorflow.keras.layers` برای ساخت مدل ها که به صورت ترتیبی با استفاده از Sequential ساخته شدند و لایه های مختلفی مانند Embedding برای بردار کردن کلمات، LSTM و Bidirectional برای یادگیری وابستگی ها و نوع مدل که Dense، BILSTM برای لایه های کاملاً متصل، و Dropout برای جلوگیری از یادگیری بیش از اندازه (Overfitting) در آن ها استفاده شده است.
12. از کتابخانه `tensorflow.keras.callbacks` از EarlyStopping و ReduceLROnPlateau به عنوان callbacks برای کنترل فرآیند آموزش مدل بهره گرفته شده است؛ که اولی آموزش را زمانی که مدل دیگر بهبود نیابد متوقف می کند، و دومی نرخ یادگیری را در صورت کاهش عملکرد تنظیم می کند.
13. از کتابخانه `tensorflow.keras` همچنین regularizers به منظور اعمال L1 و L2 برای جلوگیری از بزرگ شدن ضرایب و افزایش بیش از اندازه تعمیم پذیری مدل استفاده شده است.
14. از کتابخانه `tensorflow.keras.models` برای ساختارهای پیچیده تر، از کلاس Model استفاده می شود که به ما اجازه می دهد تا مدل هایی با چند ورودی یا چند خروجی یا مسیرهای غیرخطی طراحی کنیم که در این جهت از ان استفاده کرده ایم.
15. از کتابخانه `tensorflow.keras.layers` لایه Input برای تعریف ورودی مدل به صورت دستی و سپس برای اتصال لایه های بعدی استفاده شده است.

16. از کتابخانه `sklearn.feature_extraction.text` و `TfidfVectorizer` و `sklearn` برای تبدیل متن‌ها به بردارهای عددی استفاده کرده ایم که تکرار واژه‌ها را با توجه به اهمیت آن‌ها در اسناد مختلف در نظر می‌گیرد.

17. از کتابخانه `sklearn.model_selection` و `train_test_split` برای تقسیم داده‌ها به بخش‌های آموزش و آزمون از استفاده کرده ایم.

18. از کتابخانه `sklearn.preprocessing` و `LabelEncoder` برای تبدیل برچسب‌های متنی (مانند "مثبت"، "منفی") به مقادیر عددی (مثل صفر و یک) استفاده کرده ایم.

19. از کتابخانه `sklearn.utils.class_weight` و `compute_class_weight` در زمان آموزش مدل استفاده کرده ایم، که وزن کلاس‌ها را متناسب با فراوانی آن‌ها محاسبه کرده ایم تا مدل نسبت به کلاس‌های نادر بی تفاوت نباشد و وزن بیشتری به کلاس‌های نادرتر میدهد تا با مشکل نامتعادل بودن کلاس‌ها (class imbalance) مقابله کنیم.

20. از کتابخانه `sklearn.metrics` برای ارزیابی مدل‌ها، از `classification_report` برای محاسبه معیارهایی نظیر Accuracy، Recall و F1-score و از `confusion_matrix` برای بررسی جزئیات پیش‌بینی‌های درست و نادرست استفاده شده است.

۲-۳ بررسی و مصورسازی دیتاست‌ها

در ادامه ابتدا با کمک تکه کد زیر ادرس دیتاست‌هایی که در بخش `input` اضافه کرده ایم را بدست می‌آوریم تا آنها را وارد کرده و سپس مصورسازی کنیم.

```
# Loops through the /kaggle/input directory (where Kaggle stores uploaded datasets).
# Uses os.walk() to recursively list all files and subfolders.
# Prints the full path to each file found.
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

تصویر ۲-۳ کد بدست آوردن `directory`‌های دیتاست‌ها

خروجی کد بالا به این صورت است که در تصویر زیر مشاهده میکنید.

```

/kaggle/input/sentiment-analysis-dataset/training.1600000.processed.noemoticon.csv
/kaggle/input/sentiment-analysis-dataset/train.csv
/kaggle/input/sentiment-analysis-dataset/testdata.manual.2009.06.14.csv
/kaggle/input/sentiment-analysis-dataset/test.csv
/kaggle/input/glove-global-vectors-for-word-representation/glove.6B.200d.txt
/kaggle/input/glove-global-vectors-for-word-representation/glove.6B.50d.txt
/kaggle/input/glove-global-vectors-for-word-representation/glove.6B.100d.txt
/kaggle/input/preprocessed-dataset-sentiment-analysis/EcoPreprocessed.csv

```

تصویر ۳-۳ خروجی قطعه کد قبل

این قطعه کد به این صورت عمل میکند که به کمک دو loop فایل‌های یک دایرکتوری و زیرشاخه‌های آن را جست‌و‌جو میکند. این قطعه کد با استفاده از تابع `os.walk()` از کتابخانه‌ی `os`، تمامی پوشه‌ها و فایل‌های موجود در بخش داده‌های ورودی `'/kaggle/input'` را پیمایش می‌کند. تابع `os.walk()` به صورت بازگشتی تمام پوشه‌های فرعی را بررسی میکند که اگر این فایل‌ها در پوشه‌های مختلف داده‌ای موجود بود، ادرس آن را برگرداند و در هر مرحله، سه مقدار باز می‌گرداند که نام پوشه‌ی فعلی (`dirname`)، لیستی از زیرپوشه‌ها (که نام _ را دارد)، و لیستی از فایل‌های موجود در آن پوشه (`filenames`) در حلقه‌ی داخلی برمیگرداند، همچنین برای هر فایل موجود در لیست `filenames`، مسیر کامل آن با استفاده از دستور `os.path.join()` و سپس با کمک دستور `print` به عنوان خروجی چاپ می‌شود.

اطلاعات مربوط و ویژگی‌های این دیتاست‌ها را که به کمک مصورسازی (`visualization`) بدست آورده‌ایم، در ادامه بررسی کرده ایم. در این پروژه از سه دیتاست زیر که همگی برای حالت سه کلاسه مناسب هستند، استفاده شده است:

1. [Sentimental analysis dataset](#)
2. [Preprocessed dataset sentimental analysis](#)
3. [Glove global vectors for word representation](#)

در ادامه به ترتیب دیتاست‌ها را بررسی میکنیم تا در صورت نیاز آنها را پیش پردازش کنیم. دیتاست اول شامل دو زیربخش `train_df` و `test_df` است که ابتدا دیتاست آموزش‌شده را بررسی میکنیم برای `load` کردن دیتاست آموزش دیتاست اول از دستور خط اول استفاده کرده ام که در آن از ادرس مربوطه فایل که در قطعه کد قبل بدست آورده ام استفاده کرده ام، که در تصویر زیر قابل مشاهده است. پس از آن برای چاپ تعداد رکوردها (ردیف‌ها) و ویژگی‌ها (ستون‌ها) از دستور خط دوم و برای چاپ پنج ردیف این دیتاست از کد خط سوم استفاده کرده ام.

```
# to load the data of first dataset and print the shape of it
train_df = pd.read_csv('/kaggle/input/sentiment-analysis-dataset/train.csv', encoding='ISO-8859-1')
print(train_df.shape)
train_df.head()
```

تصویر ۳-۴ تکه کد load کردن دیتاست اول

نتیجه این قطعه کد در تصویر زیر قابل مشاهده است.

(27481, 10)

	textID	text	selected_text	sentiment	Time of Tweet	Age of User	Country	Population -2020	Land Area (Km ²)	Density (P/Km ²)
0	cb774db0d1	I'd have responded, if I were going	I'd have responded, if I were going	neutral	morning	0-20	Afghanistan	38928346	652860.0	60
1	549e992a42	Sooo SAD I will miss you here in San Diego!!!	Sooo SAD	negative	noon	21-30	Albania	2877797	27400.0	105
2	088c60f138	my boss is bullying me...	bullying me	negative	night	31-45	Algeria	43851044	2381740.0	18
3	9642c003ef	what interview! leave me alone	leave me alone	negative	morning	46-60	Andorra	77265	470.0	164
4	358bd9e861	Sons of ****, why couldn't they put them on t...	Sons of ****,	negative	noon	60-70	Angola	32866272	1246700.0	26

تصویر ۳-۵ نتیجه قطعه کد قبل

همانطور که در تصویر بالا قابل مشاهده است، که دیتاست آموزش دارای 27481 نمونه کامنت (ردیف ها) است همچنین دیتاست آموزش به ترتیب از سمت چپ به راست دارای ویژگی های (ستون ها) textID، text، sentiment، selected_text، time of tweet، age of user، country، population-2020، land area و density هستند. سپس با کمک تکه کد زیر نمودار میله ای مربوط به تعداد کامنت های هر کلاس این دیتاست را مصورسازی کرده ام.

```
# Count of each sentiment
plt.figure(figsize=(6,4))
sns.countplot(x='sentiment', data=train_df, palette='coolwarm')
plt.title('Sentiment Distribution in Training Set')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.show()
```

تصویر ۳-۶ تکه کد نمودار میله ای دیتاست آموزش دیتاست اول

در این قطعه کد، با استفاده از کتابخانه های matplotlib.pyplot و seaborn، نموداری از تعداد توییت ها بر اساس برچسب یا همان کلاس آنها، احساسات آن ها رسم کرده ام. ابتدا با استفاده از

`plt.figure(figsize=(6,4))` اندازه شکل تنظیم را مشخص کرده ام. سپس با دستور `sns.countplot()` تعداد نمونه‌های هر کلاس احساس (مثلاً «مثبت»، «منفی»، «خنثی») را در ستون `sentiment` از دیتافریم `train_df` به صورت یک نمودار میله ای (bar chart) نمایش داده ام. دستور `coolwarm` تم رنگی این نمودار را مشخص میکند. سپس با دستورات `plt.title()`, `plt.xlabel()` و `plt.ylabel()` عنوان و برچسب‌های محورهای نمودار تعیین کرده ام. در نهایت، با دستور `plt.show()` نمودار را نمایش داده ام.

نتیجه کد بالا در تصویر زیر قابل مشاهده است، در آن بیش از 10000 نمونه در کلاس خنثی (neutral)، حدود 8000 نمونه در کلاس منفی (negative) و بیشتر از 8000 نمونه در کلاس مثبت (positive) داریم.



تصویر ۳-۷ نمودار *sentimental distribution* مربوط به دیتاست اول

این نمودار برای درک میزان تعادل یا عدم تعادل بین کلاس‌های احساسی در داده‌های آموزشی بسیار مفید است چرا که اگر یک یا چندتا از این میله‌ها که نشانگر تعداد کامنت‌های آن کلاس هستند خیلی بیشتر یا کمتر از دیگر میله‌ها باشند نمایانگر *class imbalance problem* است که مستوان باعث این امر شود که مدل به اشتباه آموزش ببیند و درمورد موردهایی که به کلاس‌هایی که تعداد نمونه کمی داشته‌اند اشتباه عمل کند و نمونه‌هایی که مدل مثال‌های کمتری از آنها را دیده است به عنوان نمونه‌ای از کلاس‌های دیگر تشخیص دهد پس نوعی از تعادل باید بین تعداد نمونه‌های کلاس‌ها وجود داشته باشد که نمودار بالا دارای این هارمونی و تعادل است پس این دیتاست از این جهت نبازی به پیش پردازش با نرمالسازی ندارد پس به صورت کلی میتوان گفت که این نمودار می‌تواند در تصمیم‌گیری‌های بعدی مانند متعادل‌سازی

داده‌ها (balancing) نقش مؤثری داشته به باشد به همین جهت این روش مصورسازی را در کنار چندروش دیگر استفاده کرده ام.

سپس با تکه کد زیر تعداد و درصد داده‌های جاافتاده (missing value) را بررسی کرده ام که به این صورت است که با استفاده از دستور `isnull()` از کتابخانه `pandas`، تمامی مقادیر گمشده یعنی مقداری که `NaN` هستند را در دیتاست `train_df` شناسایی کرده ام سپس با استفاده از دستور `mean()`، نسبت مقادیر گمشده در هر ستون محاسبه کرده ام؛ چرا که در پانداس، `True` معادل 1 و `False` معادل 0 در نظر گرفته می‌شود، بنابراین میانگین آن‌ها برابر است با نسبت داده‌های گمشده که اگر این مقدار در 100 ضرب شود به درصد مقادیر جاافتاده میرسیم. نتیجه نهایی که آن را چاپ کرده ایم، درصد داده‌های گمشده در هر ستون را مشخص می‌کند. بررسی این مورد و درصد داده‌های جاافتاده از این جهت اهمیت دارد که اگر این تعداد زیاد باشد باید در مرحله پیش پردازش آنها را حذف کنیم با آنها را با مقدار مشخصی پر کنیم پس از جهت تصمیم‌گیری در مورد پاک‌سازی داده‌ها، جایگزینی مقادیر گمشده یا حذف ستون‌هایی با داده‌های بسیار ناقص اهمیت دارد.

```
# Missing data in percent
missing_percent = train_df.isnull().mean() * 100
print(missing_percent)
```

تصویر ۳-۸ کد داده‌های ناقص و جاافتاده دیتاست / آموزش دیتاست اول

که نتیجه کد بالا در تصویر زیر قابل مشاهده است، که این مقدار 0.003 درصد است یعنی از هر 10000 نمونه تنها 3 عدد دارای مقدار ناقص در دو ویژگی `text` و `selected-text` است به همین دلیل آنها را حذف نکردم.

```
textID      0.000000
text        0.003639
selected_text 0.003639
sentiment   0.000000
Time of Tweet 0.000000
Age of User  0.000000
Country      0.000000
Population -2020 0.000000
Land Area (Km²) 0.000000
Density (P/Km²) 0.000000
dtype: float64
```

تصویر ۳-۹ خروجی کد داده‌های ناقص

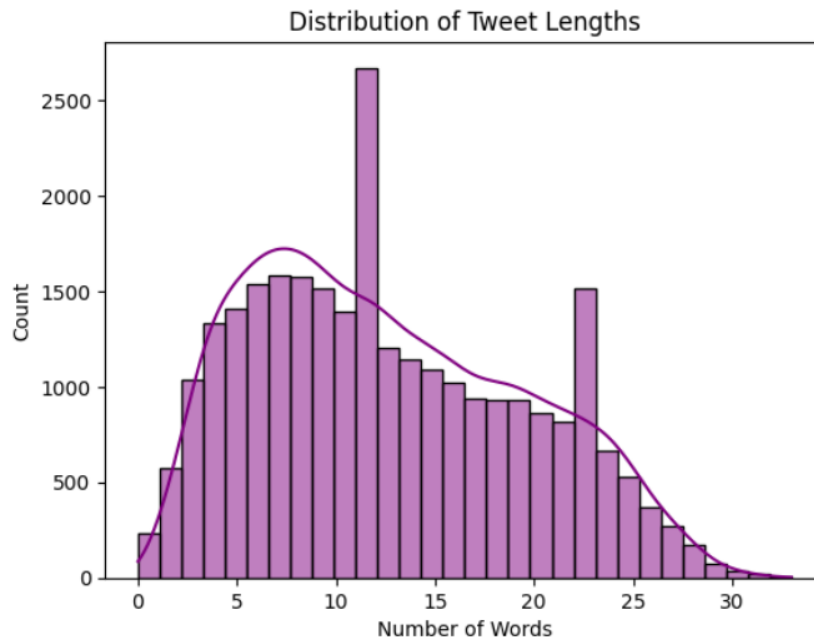
سپس با کمک تکه کد زیر این دیتاست را به نوع دیگری مصورسازی کرده ام تا از جهتی دیگر آن را بررسی کنم. این قطعه کد برای تحلیل طول توییت ها از نظر تعداد کلمات در دیتاست آموزش دیتاست اول استفاده می شود و به صورت بصری توزیع آن ها را نمایش می دهد.

```
train_df['text_length'] = train_df['text'].astype(str).apply(lambda x: len(x.split()))

sns.histplot(train_df['text_length'], bins=30, kde=True, color='purple')
plt.title("Distribution of Tweet Lengths")
plt.xlabel("Number of Words")
plt.show()
```

تصویر ۳-۱۰ کد نمودار هیستوگرام دیتاست آموزش دیتاست اول

در این کد ابتدا با افزودن یک ستون جدید به دیتافریم train_df، طول هر توییت از نظر تعداد کلمات محاسبه می شود. این کار با استفاده از astype(str) برای اطمینان از اینکه مقدار موجود در ستون text یک رشته است، و سپس با استفاده از len(x.split()) برای شمارش تعداد کلمات در هر متن توییت انجام می شود. نتیجه در ستونی به نام text_length ذخیره می شود. سپس با استفاده از sns.histplot()، یک نمودار هیستوگرام برای نمایش توزیع طول توییت ها بر حسب تعداد کلمات رسم می شود. پارامتر bins=30 برای تعیین تعداد میله های نمودار و kde=True برای افزودن خط منحنی چگالی احتمال (KDE) به نمودار استفاده شده است. رنگ نمودار نیز با مقدار color='purple' تنظیم شده است. این نمودار کمک می کند تا درک بهتری از ویژگی طول توییت ها به دست آید، که می تواند در انتخاب طول مناسب برای توالی های ورودی در مدل سازی یعنی برای padding در شبکه های LSTM مورد استفاده قرار گرفته است. این دیتاست دارای کامنت هایی با طولها و تعداد کلمات متفاوتی است که توزیع آنها به شکل زیر است:



تصویر ۳-۱۱ نمودار هیستوگرام طول کامنت‌های مربوط به دیتاست آموزش دیتاست اول

با توجه به تصویر بالا میتوان به این نتیجه رسید که تعداد کامنت‌هایی که دارای 18 کلمه هستند، بیشتر از است سپس با استفاده از تکه کد زیر این دیتاست را از جهت همبستگی ویژگی‌ها به یکدیگر بررسی کرده‌ام.

```
# Copy to avoid changing the original dataframe
df_encoded = train_df.copy()

# Convert categorical columns to numeric (excluding text if too large)
for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = df_encoded[col].astype('category').cat.codes

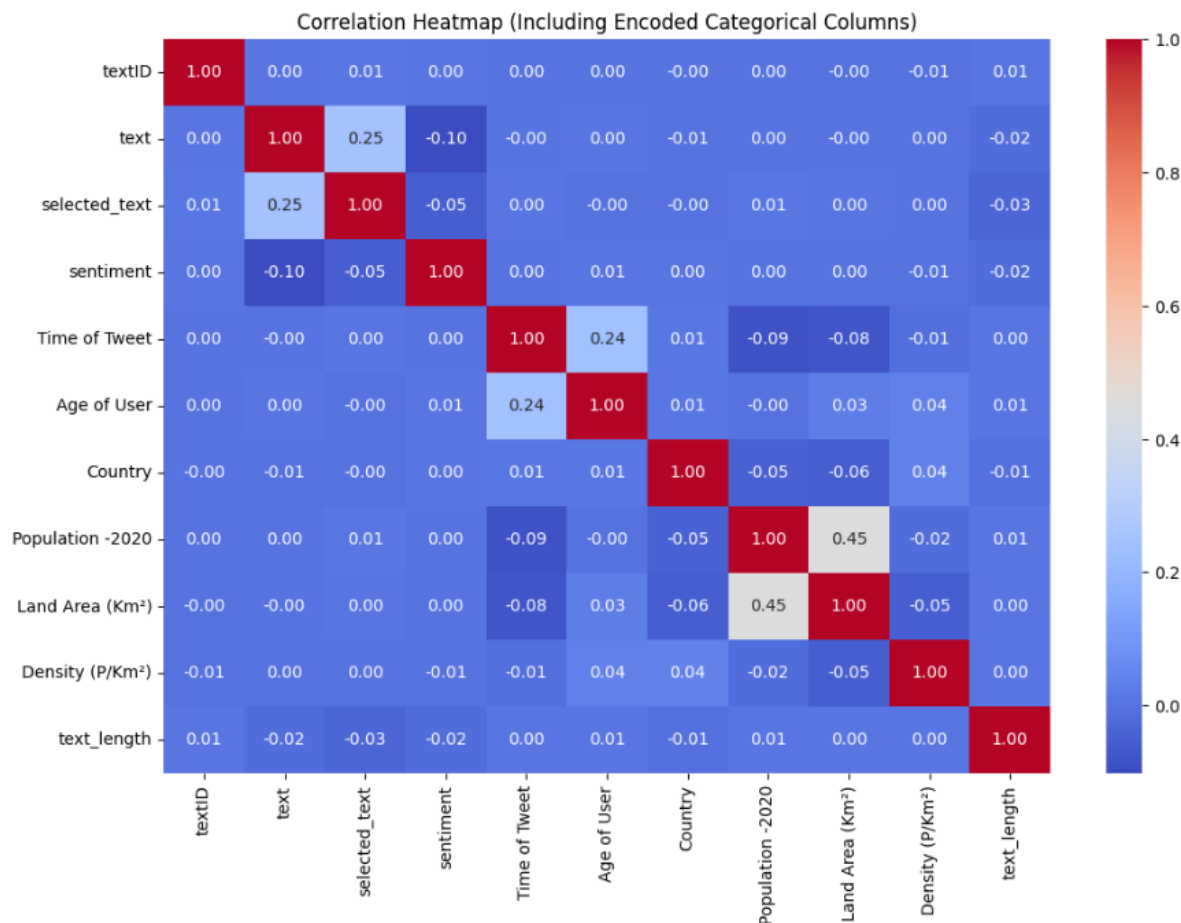
# Now plot the correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df_encoded.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Including Encoded Categorical Columns)")
plt.show()
```

تصویر ۳-۱۲ تکه کد نمودار همبستگی دیتاست آموزش دیتاست اول

همانطور که در تصویر قابل مشاهده است و کد به این صورت عمل میکند که ابتدا با ایجاد یک کپی از دیتاست train_df تحت نام df_encoded، از تغییر در داده‌های اصلی جلوگیری کرده‌ام. سپس تمامی ستون‌هایی که نوع داده‌ای آن‌ها object است (یعنی شامل داده‌های متنی یا دسته‌ای هستند)، به مقادیر عددی

تبدیل می‌شوند که این امر با استفاده از `astype('category').cat.codes` انجام داده ام که هر مقدار دسته‌ای را به یک عدد یکتا تخصیص می‌دهد. پس از آن، با استفاده از `sns.heatmap()` یک نمودار همبستگی رسم می‌شود که تنها ستون‌های عددی را در نظر می‌گیریم که این کار را با کد `numeric_only=True` انجام داده ام. این نمودار با طیف رنگی `coolwarm` و مقادیر عددی داخل سلول‌ها (`annot=True, fmt=".2f"`) نمایش داده می‌شود. نتیجه این کد در تصویر بعد قابل مشاهده است اما پیش از آنکه نتیجه آن را تحلیل کنیم لازم است بدانیم که نمودار همبستگی (Correlation Matrix) یکی از ابزارهای مهم در تحلیل داده‌هاست که رابطه‌ی خطی بین متغیرهای عددی را نمایش می‌دهد که با استفاده از ضرایب همبستگی که بین -1 تا +1 قرار دارند، نشان می‌دهد که دو متغیر تا چه حد با یکدیگر ارتباط دارند؛ به طوری که مقدار +1 بیانگر همبستگی کامل مثبت (افزایش یکی باعث افزایش دیگری می‌شود)، مقدار -1 نشان‌دهنده همبستگی کامل منفی (افزایش یکی باعث کاهش دیگری می‌شود) و مقدار صفر بیانگر نبود رابطه‌ی خطی است که در تصویر بالا که یک نمودار Heatmap است، رنگ‌های تیره‌تر همبستگی قوی‌تر را نمایش می‌دهند. این نمودار به ما کمک می‌کند از آن برای انتخاب ویژگی‌های مؤثرتر بر اساس میزان همبستگی آن‌ها با متغیر هدف استفاده کرد و کیفیت داده‌ها و ارتباط بین آن‌ها را ارزیابی کرد چراکه نمودار همبستگی کمک می‌کند تا مشخص شود کدام ویژگی‌ها رابطه‌ی خطی قوی‌تری با هم دارند و می‌تواند برای حذف ویژگی‌های تکراری یا در انتخاب ویژگی‌های ورودی مناسب به مدل‌ها بسیار مفید واقع شود که نمودار همبستگی این دیتاست همانطور که در تصویر پایین مشاهده میکنید، به این صورت است.

همانطور که در تصویر زیر مشخص است و با توجه به توضیحاتی که دادم در این دیتاست ویژگی‌ها (ستون‌ها) ارتباط همبستگی زیادی با هم ندارد به همین دلیل برای آموزش مدل تنها از دو ویژگی `text` و `sentiment` استفاده کرده ام که `text` همان متن کامنت‌ها بدون هیچگونه پیش پردازشی بر روی متن آن است و `sentiment` کلاس هر کامنت است سپس با استفاده از تکه کد زیر این دیتاست را به کمک نمودار `box-plot` مصورسازی کرده ام تا آن را بررسی کنیم.



تصویر ۳-۱۳ نمودار همبستگی correlation heatmap مربوط به دیتاست اول

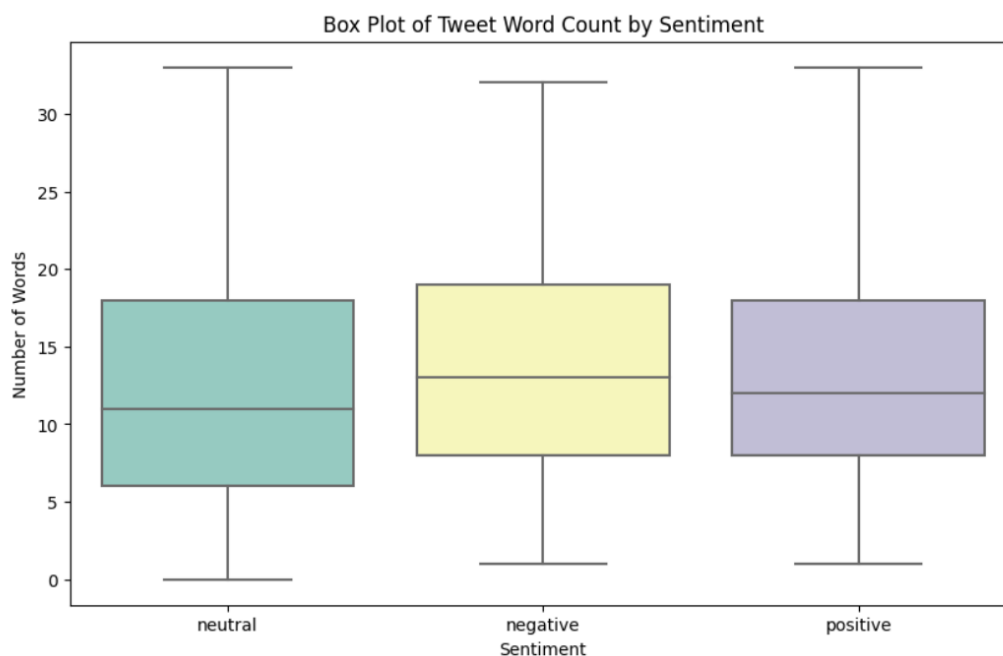
```
# Make sure tweet lengths are calculated
train_df['word_count'] = train_df['text'].fillna('').apply(lambda x: len(x.split()))

# Box plot of word count
plt.figure(figsize=(10, 6))
sns.boxplot(data=train_df, x='sentiment', y='word_count', palette='Set3')
plt.title('Box Plot of Tweet Word Count by Sentiment')
plt.xlabel('Sentiment')
plt.ylabel('Number of Words')
plt.show()
```

تصویر ۳-۱۴ کد box-plot مربوط به دیتاست آموزش دیتاست اول

ابتدا یک ستون اضافه کرد ام که از روی ستون text، طول توییت‌ها از نظر تعداد کلمات (word_count) در دیتافریم train_df محاسبه کرده ام که در صورت وجود داده‌های گمشده در ستون text، این مقادیر با

رشته‌ی خالی جایگزین کرده‌ام تا خطایی در محاسبات رخ ندهد. سپس، تعداد کلمات هر توییت با استفاده از تابع `split()` محاسبه شده و در ستون `word_count` ذخیره کرده‌ام. در مرحله بعد، با استفاده از تابع `sns.boxplot()` نمودار جعبه‌ای (box-plot) رسم کرده‌ام که در آن محور افقی (x) نشان‌دهنده نوع احساس (مثبت، منفی، خنثی) و محور عمودی (y) نشان‌دهنده تعداد کلمات کامنت‌ها است. این نمودار به خوبی می‌تواند میانه (median)، چارک‌ها (quartiles)، و نقاط پرت (outliers) را برای هر کلاس احساسی نمایش دهد. نمودار box-plot این دیتاست در تصویر زیر قابل مشاهده است.



تصویر ۳-۱۵ نمودار box-plot مربوط به کلاس‌های دیتاست آموزش دیتاست اول

برای آنکه بتوانیم این نمودار را تحلیل کنیم باید ابتدا این نمودار را بشناسیم. نمودار Box Plot (یا نمودار جعبه‌ای) یکی از ابزارهای مهم در آمار و تحلیل داده است که برای نمایش توزیع داده‌ها و شناسایی نقاط پرت (Outliers) به کار می‌رود. این نمودار با استفاده از پنج عدد خلاصه (مقدار حداقل، چارک اول Q1، میانه، چارک سوم Q3، و مقدار حداکثر) که تصویری ساده اما دقیق از پراکندگی و تمرکز داده‌ها ارائه می‌دهد. بخش اصلی این نمودار یک جعبه است که از چارک اول تا چارک سوم را در بر می‌گیرد و نشان‌دهنده 50 درصد میانی داده‌هاست، در حالی که خط میانه درون جعبه موقعیت مرکزی داده‌ها را نمایش می‌دهد. دو خط در دو طرف جعبه، محدوده داده‌هایی را که خارج از چارک‌ها اما هنوز در دامنه طبیعی قرار دارند، مشخص می‌کنند. نقاطی که خارج از این محدوده باشند به عنوان داده‌های پرت با

علامت‌های جداگانه نمایش داده می‌شوند. استفاده از نمودار جعبه‌ای برای تحلیل داده‌های متنی از این جهت مفید است که می‌توان دریافت آیا توییت‌های دارای احساس خاصی معمولاً طولانی‌تر یا کوتاه‌تر هستند و آیا توزیع طول آن‌ها گسترده یا محدود است پس با توجه به تصویر و اطلاعات داده شده در رابطه با این نوع نمودار می‌توانیم به این نتیجه برسیم که اکثریت کامنت‌ها دارای تعداد کلمات به این صورت هستند، کلاس neutral، در بازه حدود 6 تا 18 کلمه، کلاس negative، در بازه حدودی 8 تا 19 کلمه و کلاس positive، در بازه حدودی 8 تا 17 کلمه است.

همانطور که پیش‌تر به آن اشاره کردیم دیتاست اول تشکیل شده از دو دیتاست تست و آموزش است که دیتاست آموزش را بررسی کردیم حال به بررسی دیتاست تست این دیتاست می‌پردازیم تا آن را با استفاده از همان تکنیک‌های قبل بررسی کنیم و در صورت نیاز پیش‌پردازش بر روی آن انجام دهیم.

این دیتاست تست دیتاست اول را با استفاده از کد زیر استفاده کرده‌ام که درست مشابه تکه کدی است که به کمک آن ساختار کلی دیتاست آموزش را چاپ کرده بودم. در این کد، ابتدا با استفاده از تابع `pd.read_csv()` فایل `test.csv` از مسیر `/kaggle/input/sentiment-analysis-dataset/` دسترسی پیدا کرده‌ام و آن را چاپ کرده‌ام. پارامتر `encoding='ISO-8859-1'` برای اطمینان از خواندن صحیح کاراکترهای خاص (مانند علائم نگارشی یا حروف غیر انگلیسی) تعیین شده است، چرا که گاهی فایل‌های CSV به صورت پیش‌فرض با کدگذاری UTF-8 سازگار نیستند. پس از بارگذاری، با استفاده از دستور `print(test_df.shape)` ابعاد دیتافریم شامل تعداد سطرها (نمونه‌ها) و ستون‌ها نمایش داده‌ام. در نهایت، `test_df.head()` پنج ردیف اول دیتافریم را نمایش داده‌ام. این دیتاست تست نسبت به دیتاست آموزش یک ویژگی کمتر دارد که آن هم `selected-text` است که حالت پیش‌پردازش شده ویژگی `text` بود.

```
#visualization of test dataset
test_df = pd.read_csv('/kaggle/input/sentiment-analysis-dataset/test.csv', encoding='ISO-8859-1')
print(test_df.shape)
test_df.head()
```

تصویر ۳-۱۶ کد نمایش کلی دیتاست تست دیتاست اول

این دیتاست تست دیتاست اول دارای 4815 نمونه کامنت است ویژگی‌ها و نمونه‌های مربوط به دیتاست تست دیتاست اول همانطور که به آن اشاره کردیم در شکل زیر قابل مشاهده است.

(4815, 9)

	textID	text	sentiment	Time of Tweet	Age of User	Country	Population -2020	Land Area (Km ²)	Density (P/Km ²)
0	f87dea47db	Last session of the day http://twitpic.com/67ezh	neutral	morning	0-20	Afghanistan	38928346.0	652860.0	60.0
1	96d74cb729	Shanghai is also really exciting (precisely ~...	positive	noon	21-30	Albania	2877797.0	27400.0	105.0
2	eee518ae67	Recession hit Veronique Branquinho, she has to...	negative	night	31-45	Algeria	43851044.0	2381740.0	18.0
3	01082688c6	happy bday!	positive	morning	46-60	Andorra	77265.0	470.0	164.0
4	33987a8ee5	http://twitpic.com/4w75p - I like it!!	positive	noon	60-70	Angola	32866272.0	1246700.0	26.0

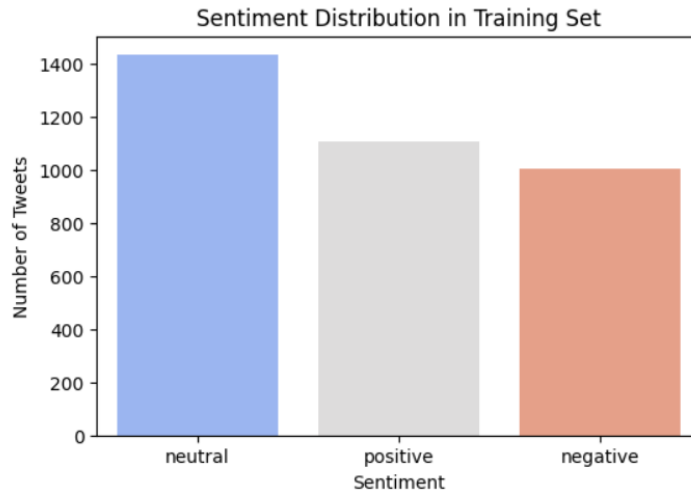
تصویر ۳-۱۷ دیتاست تست دیتاست اول

سپس با کمک تکه کد زیر نمودار میله ای مربوط به تعداد کامنت های هر کلاس این دیتاست را مصورسازی کرده ام.

```
# Count of each sentiment
plt.figure(figsize=(6,4))
sns.countplot(x='sentiment', data=test_df, palette='coolwarm')
plt.title('Sentiment Distribution in Training Set')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.show()
```

تصویر ۳-۱۸ کد نمودار میله ای دیتاست تست دیتاست اول

این قطعه کد یک نمودار میله ای از توزیع احساسات (Sentiments) از دیتاست test_df ترسیم می کند که همان دیتاست تست دیتاست اول میباشد. ابتدا با دستور plt.figure(figsize=(6,4)) اندازه و ابعاد نمودار را تعیین میکنیم. سپس با استفاده از sns.countplot()، تعداد نمونه های هر کلاس احساس را در محور X نمایش میدهم؛ پارامتر data=test_df مشخص می کند که داده ها از کدام دیتافریم یا دیتاست گرفته شوند و palette='coolwarm' پالت رنگی را برای میله ها مشخص می کند. در ادامه، با plt.xlabel()، plt.title() و plt.ylabel() به ترتیب عنوان نمودار و برچسب محورهای X و Y را مشخص میکنیم. در نهایت، با دستور plt.show() نمودار را نمایش می دهیم. نتیجه کد بالا در تصویر زیر قابل مشاهده است.



تصویر ۳-۱۹ نمودار میله ای کلاس هاس دیتاست تست دیتاست اول

همانطور که پیش تر گفتیم این نمودار برای درک میزان تعادل یا عدم تعادل بین کلاس های احساسی در دیتاست ها بسیار مفید است چراکه اگر یک یا چندتا از این میله ها که نمایانگر تعداد کامنت های آن کلاس هستند خیلی بیشتر یا کمتر از دیگر میله ها باشند نمایانگر class imbalance problem است که مستوان باعث این امر شود که مدل به اشتباه آموزش ببیند و درمورد موردهایی که به کلاس هایی که تعداد نمونه کمی داشته اند اشتباه عمل کند. پس نوعی از تعادل باید بین تعداد نمونه های کلاس ها وجود داشته باشد که نمودار بالا دارای این هارمونی و تعادل است پس این دیتاست از این جهت نبازی به پیش پردازش با نرمالسازی ندارد.

```
# Missing data in percent for test dataset
missing_percent = test_df.isnull().mean() * 100
print(missing_percent)
# which means for this amount of records 4815 we have 1281 missing value that can be dropped
# tahn will remain 3534, we can only drop the records which have a missing row of text
# cause others can be predicted or doesnt matter for us
# to Drop rows where 'text' column is missing (since it's essential for NLP tasks)

initial_shape = test_df.shape
test_df = test_df.dropna(subset=['text'])
final_shape = test_df.shape

print(f"\nDropped rows with missing 'text': {initial_shape[0] - final_shape[0]} rows removed.")
print(f"Remaining rows: {final_shape[0]}")
```

تصویر ۳-۲۰ کد تشخیص و حذف داده های ناقص مربوط به دیتاست تست دیتاست اول

در این کد به بررسی و حذف مقادیر گمشده (Missing Values) در دیتافریم `test_df` می‌پردازیم، با تمرکز بر ستون 'text' می‌باشد. ابتدا، با استفاده از `test_df.isnull().mean() * 100` درصد مقادیر گمشده در هر ستون محاسبه کرده‌ام؛ که در واقع این مقدار میانگین مقادیر NaN برای هر ستون است که در درصد ضرب کرده‌ام تا درصد داده‌های گمشده و ناقص مشخص شود. سپس این مقدار را چاپ کردم، که نسبت داده‌های ناقص هستند. چون ستون 'text' برای انجام تحلیل لازم و ضروری است، تمرکز بیشتر بر روی ردیف‌هایی که مقدار آن در ستون 'text' گم شده است، پس با دستور `test_df.dropna(subset=['text'])` ستون‌ها را حذف کرده‌ام. مقدار اولیه و نهایی شکل دیتافریم با دستور `shape` ذخیره کرده‌ام تا مشخص شود چند ردیف حذف شده‌اند و بتوانیم آنها را چاپ کنیم. که در نهایت، تعداد ردیف‌های حذف‌شده و باقی‌مانده چاپ کرده‌ام. این کار باعث می‌شود داده‌های نامناسب برای تحلیل متنی حذف شده و کیفیت مجموعه داده برای آموزش مدل حفظ شود. نتیجه کد بالا در تصویر زیر قابل مشاهده است.

```
textID      26.604361
text        26.604361
sentiment   26.604361
Time of Tweet  26.604361
Age of User   26.604361
Country      26.604361
Population -2020 26.604361
Land Area (Km²) 26.604361
Density (P/Km²) 26.604361
dtype: float64

Dropped rows with missing 'text': 1281 rows removed.
Remaining rows: 3534
```

تصویر ۳-۲۱ داده‌های ناقص و نتیجه حذف آنها از دیتاست تست دیتاست اول

در نتیجه کد بالا از 4815 ردیف، 1281 ردیف مقدار گمشده دارند، که بعد از حذف کردن آنها، 3534 ردیف باقی می‌مانند. همچنین مشاهده می‌کنیم که درصد مقادیر ناقص تمامی ستون‌ها برابر است که نشان دهنده آن است که تعدادی ردیف در دیتاست داریم که در آنها تمامی این مقاداری و ویژگی‌ها جافتاده‌اند و ناقص هستند و این تعداد نسبت به دیتاست آموزش دیتاست اول بسیار بیشتر است که 26 درصد است به همین خاطر آنها را حذف کرده‌ام.

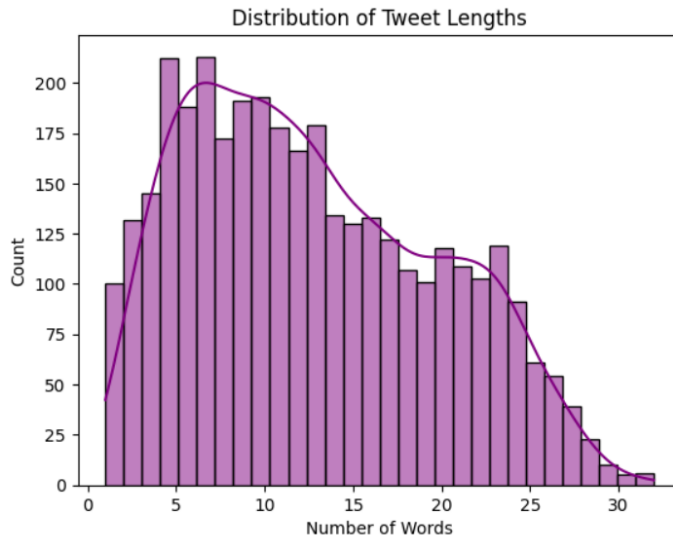
سپس با استفاده از کد زیر این دیتاست را از جهت تعداد کلمات و طول کامنت‌ها بررسی کرده‌ام.

```
test_df['text_length'] = test_df['text'].astype(str).apply(lambda x: len(x.split()))

sns.histplot(test_df['text_length'], bins=30, kde=True, color='purple')
plt.title("Distribution of Tweet Lengths")
plt.xlabel("Number of Words")
plt.show()
```

تصویر ۳-۲۲ کد نمودار طول کامنت های دیتاست تست دیتاست اول

در این کد طول متن کامنت ها (یعنی تعداد کلمات هر کامنت) را محاسبه کرده ام و سپس توزیع آن را با استفاده از هیستوگرام و منحنی چگالی نمایش داده ام. در خط اول، با دستور `test_df['text'].astype(str)` اطمینان حاصل کرده ام که مقدار ستون 'text' به فرمت رشته (string) است. سپس با دستور `apply(lambda x: len(x.split()))` تعداد کلمات در هر توییت محاسبه کرده ام همچنین با کمک دستور `x.split()` متن را با استفاده از فاصله های موجود به لیستی از کلمات تقسیم کرده ام و با استفاده از دستور `len()` تعداد عناصر این لیست (یعنی تعداد کلمات) را برگردانده ام و این مقادیر در یک ستون جدید به نام 'text_length' ذخیره کرده ام تا سپس و در ادامه، با استفاده از دستور `sns.histplot()` یک هیستوگرام از این ستون بدست آورده رسم کرده ام که بخش `test_df['text_length']` ورودی نمودار است که شامل طول های متنی تمام کامنت هاست که آنها را محاسبه کرده ایم همچنین `bins=30` تعداد میله های هیستوگرام است که مشخص کرده ایم همچنین با استفاده از پارامتر `kde=True` منحنی چگالی Kernel Density Estimate را فعال و نمایش داده ام که همان خط تیره رنگی است که در تصویر مشاهده میشود. پارامتر `color='purple'` رنگ نمودار را بنفش تعیین می کند. در نهایت، با استفاده از دستور `plt.title()` عنوان نمودار، با استفاده از دستور `plt.xlabel()` برچسب محور x و با استفاده از دستور `plt.show()` خود نمودار را روی صفحه نمایش داده ام. نتیجه این کد در تصویر زیر قابل مشاهده است که همانطور که مشخص است، این دیتاست تست دیتاست اول دارای کامنت هایی با طولها و تعداد کلمات متفاوتی است که همانطور که مشاهده میکنید، تعداد زیادی از کامنت ها دارای طول 4 تا 13 کلمه هستند و باقی کامنت ها دارای طول های متفاوتی هستند که این طول ها تقریباً با هم برابرند و دارای هارمونی و تعادل خاصی هستند از این جهت که هیچ یک از این بازه ها خیلی بیشتر یا خیلی کمتر از دیگر طول ها نیست.



تصویر ۳-۲۳ نمودار هیستوگرام مربوط به دیتاست تست دیتاست اول

سپس با استفاده از قطعه کد زیر نمودار همبستگی این دیتاست تست دیتاست اول را به نمایش گذاشته ام.

```
# Copy to avoid changing the original dataframe
df_encoded = test_df.copy()

# Convert categorical columns to numeric (excluding text if too large)
for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = df_encoded[col].astype('category').cat.codes

# Now plot the correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(df_encoded.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Including Encoded Categorical Columns)")
plt.show()
```

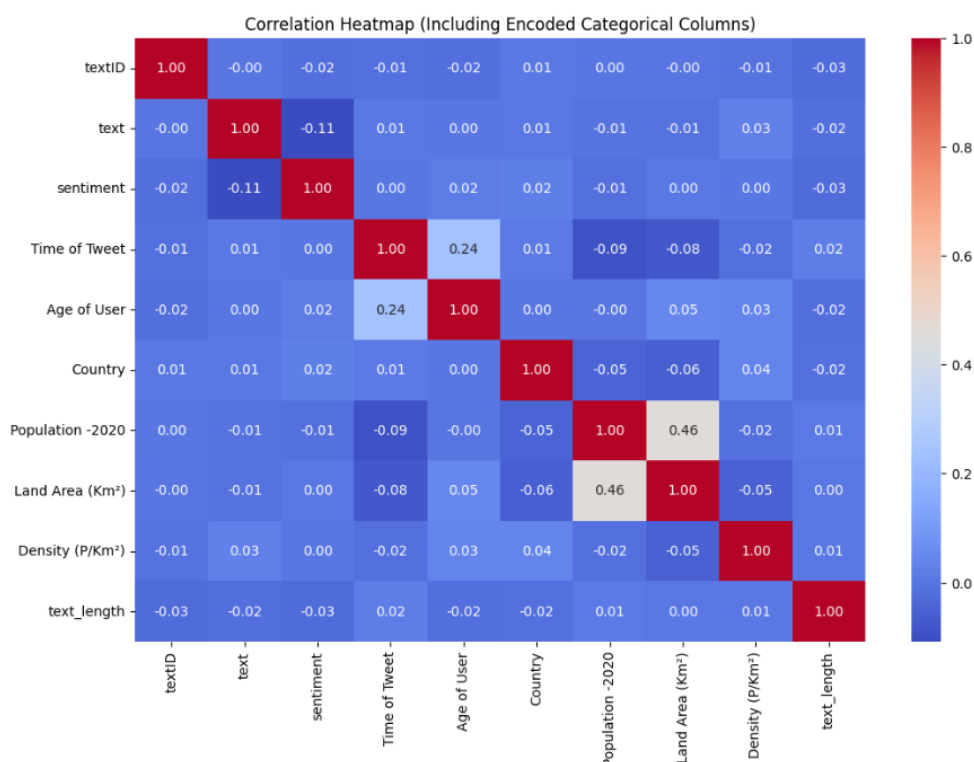
تصویر ۳-۲۴ کد مربوط به نمودار همبستگی دیتاست تست دیتاست اول

در این کد، ابتدا یک نسخه‌ی کپی از دیتافریم `test_df` ایجاد کرده ام و سپس تمامی ستون‌های دسته‌ای (categorical) را به مقادیر عددی تبدیل کرده ام تا بتوانم یک نمودار همبستگی از این دیتاست و همبستگی بین ویژگی‌ها را به نمایش بگذارم. پس با استفاده از دستور `df_encoded = test_df.copy()` یک کپی از دیتافریم ساخته ام تا تغییراتی که در ادامه اعمال می‌کنم، روی دیتافریم اصلی اثر نگذارد. سپس، با یک حلقه‌ی `for`، تمام ستون‌های دیتافریم بررسی کرده ام تا اگر نوع داده‌ی یک ستون `object` باشد که معمولاً نشان‌دهنده داده‌های متنی یا دسته‌ای است، با استفاده از دستور `astype('category').cat.codes` آن ستون به کدهای عددی (integer codes) تبدیل کرده ام.

این تبدیل کردن به کدهای عددی به ما این اجازه را می‌دهد تا بتوانیم از توابع آماری مثل محاسبه همبستگی بین ویژگی‌ها استفاده کنیم؛ چرا که این توابع تنها با داده‌های عددی کار می‌کنند و در غیر این صورت نمیتوانیم این نمودار را رسم کنیم. در بخش بعدی، با استفاده از دستور `plt.figure(figsize=(12, 8))` اندازه شکل و نمودار را تنظیم کردم. سپس با استفاده از دستور `sns.heatmap()` یک نمودار از همبستگی بین ستون‌های عددی رسم کرده‌ام.

پارامتر `df_encoded.corr(numeric_only=True)` فقط ستون‌های عددی را در نظر می‌گیرد و ماتریس همبستگی آن‌ها را محاسبه می‌کند. `annot=True` باعث می‌شود مقادیر عددی همبستگی داخل هر سلول نوشته شود. پارامتر `cmap='coolwarm'` پالت رنگی را برای نمایش شدت همبستگی‌ها انتخاب‌ها را مشخص کرده‌ام.

`fmt=".2f"` نمایش اعداد را به دو رقم اعشار محدود می‌کند. در نهایت، با استفاده از دستور `plt.title()` عنوانی برای نمودار مشخص کرده‌ام و با استفاده از دستور `plt.show()` نمودار را نمایش داده‌ام. نمودار همبستگی این دیتاست همانطور که در تصویر پایین مشاهده میکنید، به این صورت است.



تصویر ۳-۲۵ نمودار `correlation heatmap` مربوط به دیتاست تست دیتاست اول

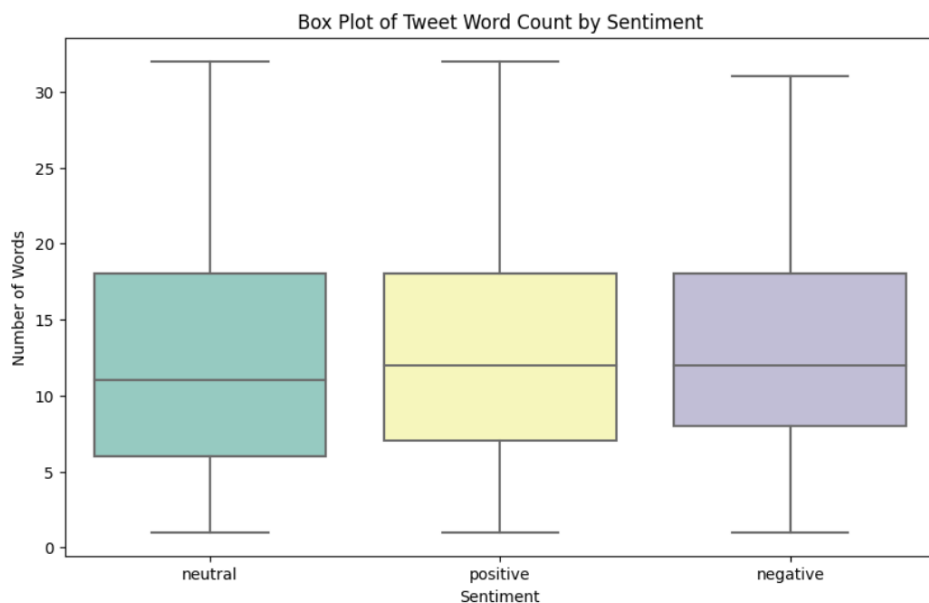
سپس با استفاده از قطعه کد زیر نمودار box-plot این دیتاست تست دیتاست اول را به نمایش گذاشته ام تا بتوانم داده های پرت را پیدا کنم و در صورت نیاز پیش پردازش های لازم را انجام دهم و حذف اشان کنم.

```
# Make sure tweet lengths are calculated
test_df['word_count'] = test_df['text'].fillna('').apply(lambda x: len(x.split()))

# Box plot of word count
plt.figure(figsize=(10, 6))
sns.boxplot(data=test_df, x='sentiment', y='word_count', palette='Set3')
plt.title('Box Plot of Tweet Word Count by Sentiment')
plt.xlabel('Sentiment')
plt.ylabel('Number of Words')
plt.show()
```

تصویر ۳-۲۶ قطعه کد نمودار box-plot دیتاست تست دیتاست اول

ابتدا در این کد، با استفاده از دستور `test_df['text'].fillna("")` اطمینان حاصل کرده ام که هیچ مقدار گم شده ای (NaN) در ستون 'text' وجود ندارد، زیرا اگر داده ای گم شده باشد، می تواند در محاسبات خطا ایجاد کند. در خط بعد، تعداد کلمات در هر توییت را با استفاده از دستور `split()` محاسبه کرده ام که این تابع متن را با استفاده از فاصله های موجود در کامنت ها، آنها را به لیستی از کلمات تقسیم می کند، و سپس با استفاده از دستور `len()` تعداد این کلمات را برگردانده ام که پس از آن نتیجه را در ستونی به نام 'word_count' ذخیره کرده ام. در بخش دوم، با استفاده از دستور `plt.figure(figsize=(10, 6))` اندازه نمودار را تعیین و مشخص کرده ام. سپس با استفاده از دستور `sns.boxplot()` نمودار box-plot را به نمایش در آورده ام. پارامتر های این قطعه کد از این جهت دارای اهمیت هستند که پارامتر `data=test_df` مشخص می کند که داده ها از کدام دیتافریم گرفته شود، پارامتر `x='sentiment'` محور افقی را به دسته های احساسی یعنی همان کلاس های هدف تقسیم کرده است، پارامتر `y='word_count'` محور عمودی را بر اساس تعداد کلمات کامنت ها تشکیل می دهد و تقسیم و تعیین میکند و پارامتر `palette='Set3'` پالت رنگی را برای هر گروه احساسی مشخص میکند. در نهایت، برای تعیین عنوان نمودار و همچنین برچسب های محور X و Y از دستور های `plt.title()`، `plt.xlabel()` و `plt.ylabel()` استفاده کرده ام و سپس با استفاده از دستور `plt.show()` نمودار را به نمایش گذاشته ام. نتیجه ایم کد در تصویر زیر مشخص است که آن را در ادامه بررسی کرده ام.



تصویر ۲۷-۳ نمودار box-plot مربوط به دیتاست تست دیتاست اول

با توجه به این نوع نمودار میتوانیم به این نتیجه برسیم که اکثریت کامنت‌ها دارای تعداد کلمات به این صورت هستند، کلاس neutral، در بازه حدود 6 تا 18 کلمه، کلاس negative، در بازه حدودی 8 تا 18 کلمه و کلاس positive، در بازه حدودی 7 تا 18 کلمه است همچنین هیچ داده پرتی در این دیتاست مشاهده نمی‌شود.

حال به بررسی دیتاست دوم میپردازیم که برخلاف دیتاست قبل از دو بخش تست و آموزش تشکیل نشده است. این دیتاست را به کمک تکه کدی مشابه کدهای قبلی که در تصویر زیر قابل مشاهده است به تصویر در آورده‌ام و برای دو دیتاست پیشین نحوه عملکرد این کد را شرح داده‌ام به جهت جلوگیری از تکرار مکررات دیگران را شرح نخواهم داد چرا که تنها امری که متفاوت است دایرکتوری وارد شده می‌باشد.

```
prepro = pd.read_csv('/kaggle/input/preprocessed-dataset-sentiment-analysis/EcoPreprocessed.csv', encoding='ISO-8859-1')
print(prepro.shape)
prepro.head()
```

تصویر ۲۸-۳ کد مصورسازی اولیه مربوط به دیتاست دوم

نتیجه کد تصویر بالا، در تصویر زیر قبل مشاهده است که این دیتاست دارای 4084 نمونه کامنت (ردیف ها) است همچنین این دیتاست به ترتیب از سمت چپ به راست دارای ویژگی های (ستون ها) index، review، polarity و division است که کلاس های هدف ما ویژگی division هستند.

(4084, 4)

	Unnamed: 0	review	polarity	division
0	3870	able play youtube alexa	0.5000	positive
1	62	able recognize indian accent really well drop ...	0.2794	positive
2	487	absolute smart device amazon connect external ...	0.1827	positive
3	3204	absolutely amaze new member family control hom...	0.3682	positive
4	1265	absolutely amaze previously sceptical invest m...	0.2333	positive

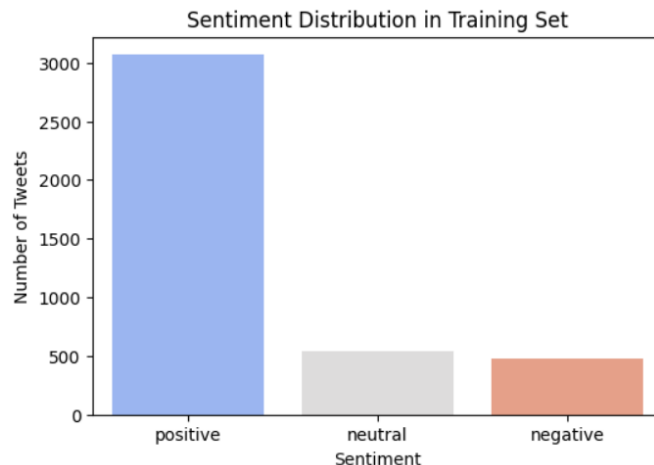
تصویر ۳-۲۹ دیتاست دوم

سپس با استفاده از قطعه کد زیر نمودار میله ای این دیتاست را به نمایش درآورده ام که مشابه کدی است که به همین جهت برای دو دیتاست قبلی استفاده کرده بودم.

```
# Count of each sentiment
plt.figure(figsize=(6,4))
sns.countplot(x='division', data= prepro, palette='coolwarm')
plt.title('Sentiment Distribution in Training Set')
plt.xlabel('Sentiment')
plt.ylabel('Number of Tweets')
plt.show()
```

تصویر ۳-۳۰ کد نمودار میله ای مربوط به دیتاست دوم

نتیجه این کد را در تصویر زیر مشاهده میکنید که بر اساس آن میتوان گفت که این دیتاست همانطور که در تصویر زیر مشاهده میکنید، دارای حدود 400 کامنت در کلاس neutral، حدود 500 کامنت در کلاس negative و حدود 3000 کامنت در کلاس positive میباشد.



تصویر ۳-۳۱ نمودار میله ای دیتاست دوم

در تصویر بالا مشاهده میکنیم که اختلاف زیاد و قابل توجهی در کلاس های neutral و negative در مقایسه با کلاس positive وجود دارد که این اختلاف و عدم وجود تعادل و هارمونی بین تعداد کامنت هاب کلاس های دیگر میتواند باعث آموزش نادرست مدل شود و باعث این امر شود که مدل به سمت تشخیص داده ها به عنوان کلاس positive، bias داشته باشد. برای حل مشکل class imbalance problem به دو روش میتوان عمل کرد در روش اول میتوان با روش های oversampling یا undersampling و با کمک یک قطعه کد تعداد رکوردهای سه کلاس را با هم برابر کنیم که به این معنا هستند که در روش undersampling از کلاسی که بیشترین تعداد رکورد را دارد (کلاس ماژور) تعدادی از نمونه ها را به صورت رندوم و شانسی انتخاب میکنیم که تعداد نمونه هایی که انتخاب میکنیم باید با تعداد نمونه و رکورد های کلاس های هدف دیگر در تعادل باشد یعنی تعداد رکورد های این کلاس را کاهش میدهیم تا به تعدادی متعادل با کلاسی که نمونه ها و رکورد های کمتری دارد (کلاس مینور) برسد و در روش oversampling به صورت رندوم و شانسی از نمونه های موجود در کلاس هایی که تعداد رکورد و سمپل کمتری دارد نمونه بر میداریم و به آن کلاس ها اضافه میکنیم یعنی در واقع با این روش تعداد رکورد های کلاس های مینور را افزایش میدهیم تا به تعدادی متعادل با کلاس ماژور برسند.

یا با روش دوم در مرحله آموزش مدل از متودی استفاده کنیم که در آن به صورت خود کار به کلاس های مینور ضریب بالاتری اختصاص داده میشود در حالیکه کلاس های ماژور ضریب پایین تری دارند، تا از این طریق مدل متوجه اهمیت این کلاس ها بکنیم و مانع بایاس پیدا کردن مدل شویم که مت در این پروژه از

روش دوم استفاده کرده ام چرا که دیتاست مورد نظر دارای حجم بسیار زیادی از رکورد نیست پس باید در صورت استفاده از روش اول، باید از روش oversampling استفاده میکردیم چرا که برای دیتاست هایی که دارای تعداد بالایی رکورد هستند روش undersampling پیشنهاد میشود (چرا که این روش تعداد رکوردها را محدودتر و کمتر میکند و دیتاست آموزش نباید تعداد خیلی کمی رکورد داشته باشد تا مدل به حد کافی نمونه ببیند و الگوها را به درستی بیاموزد) من در این پروژه از هر دو روش استفاده کرده ام به این صورت که پیش از استفاده از دیتاست دوم کلاس های آن را به کمک oversampling بالانس کردم و همچنین در فرایند آموزش مدل از روش دوم نیز استفاده کرده ام که در ادامه و در قطعه کدهای بعدی آن را بررسی خواهیم کرد.

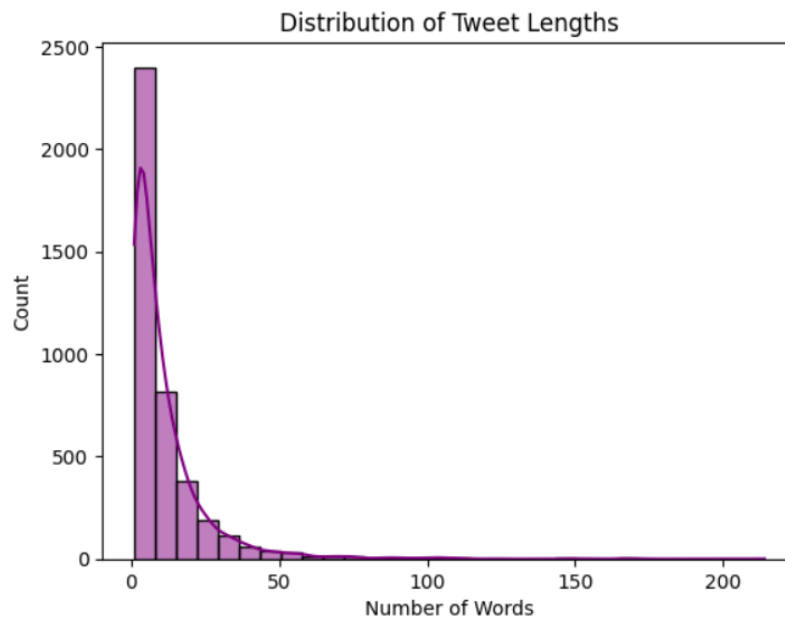
سپس با استفاده از کد زیر که مشابه کدهای نمودار هیستوگرام پیشین است نمودار هیستوگرام این دیتاست را به نمایش گذاشته ام. این کد درست مشابه کد هیستوگرام دو دیتاست پیشین است به همین جهت از توضیح مکرر آن خودداری میکنم.

```
prepro['text_length'] = prepro['review'].astype(str).apply(lambda x: len(x.split()))

sns.histplot(prepro['text_length'], bins=20, kde=True, color='purple')
plt.title("Distribution of Tweet Lengths")
plt.xlabel("Number of Words")
plt.show()
```

تصویر ۳-۳۲ کد مربوط به نمودار هیستوگرام دیتاست دوم

نتیجه کد بالا را میتوان در تصویر زیر مشاهده کرد که همانطور که پیش تر به آن اشاره کردیم، این نمودار توزیع طول کامنت ها بر اساس تعداد کلمات را نشان می دهد. محور افقی تعداد کلمات از 0 تا 200 است و محور عمودی تعداد کامنت ها را نمایش می دهد. بیشترین فراوانی کامنت ها در محدوده 0 تا 50 کلمه است، و البته یک اوج در حدود 2500 کامنت برای کامنت های کوتاه تر را داریم که البته با افزایش تعداد کلمات، تعداد توئیت ها به طور قابل توجهی کاهش می یابد، به طوری که برای توئیت های با طول بیش از 100 کلمه، تعداد توئیت ها به کمتر از 500 می رسد. این الگو نشان دهنده ترجیح کاربران برای نوشتن توئیت های مختصر و کوتاه است.



تصویر ۳-۳۳ نمودار تعداد کلمات و تعداد کامنت های دیتاست دوم

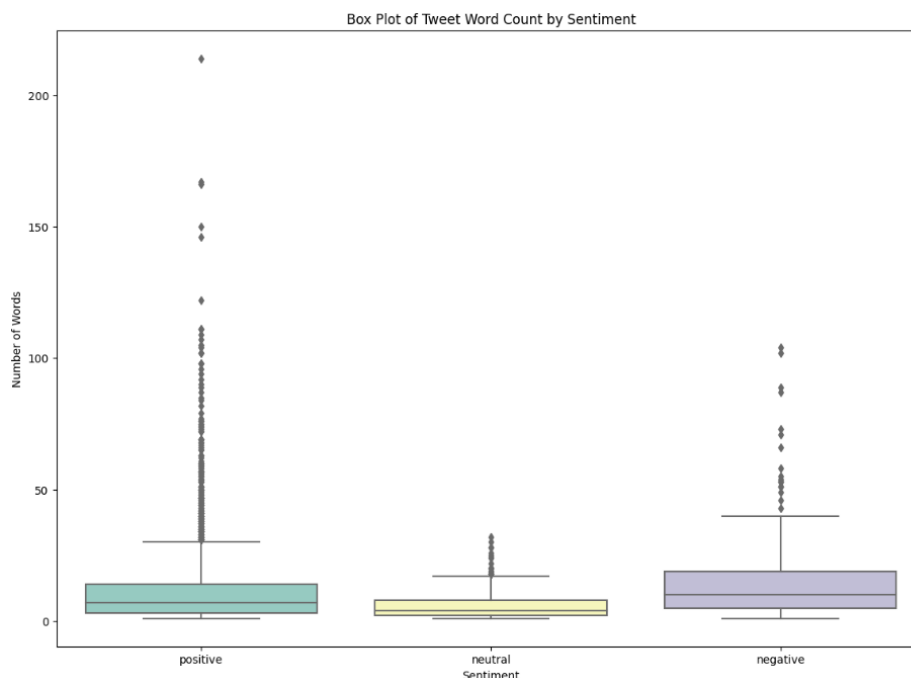
سپس با استفاده از کد زیر که مشابه کدهای نمودار box-plot پیشین است این نمودار را برای این دیتاست به نمایش گذاشته ام. این کد درست مشابه کد نمودار box-plot دو دیتاست پیشین است به همین جهت از توضیح مکرر آن خودداری میکنم.

```
# Make sure tweet lengths are calculated
prepro['word_count'] = prepro['review'].fillna('').apply(lambda x: len(x.split()))

# Box plot of word count
plt.figure(figsize=(10, 6))
sns.boxplot(data=prepro, x='division', y='word_count', palette='Set3')
plt.title('Box Plot of Tweet Word Count by Sentiment')
plt.xlabel('Sentiment')
plt.ylabel('Number of Words')
plt.show()
```

تصویر ۳-۳۴ کد نمودار box-plot دیتاست دوم

نتیجه کد بالا را میتوان در تصویر زیر مشاهده کرد که در ادامه آن را بررسی و تحلیل کرده ام.



تصویر ۳-۳۵ نمودار box-plot دیتاست دوم پیش از پیش پردازش

از نمودار بالا میتوان این نتیجه را گرفت که در این دیتاست تعداد زیادی از کامنت ها با طول تعداد کلمات خارج از بازه نرمال داریم که به عنوان داده پرت تلقی میشوند که این داده ها را به دو صورت میتوان کنترل کرد که در روش اول میتوان با استفاده از یک قطعه کد و با کمک چارک های اول و سوم و ضریب IQR این داده ها را حذف کرد اما این کار دارای ایراداتی است ان هم اینکه ممکن است این داده هایی که طول بیشتری دارند و به عنوان داده پرت تلقی شده اند دارای اطلاعات مهمی باشند که این عمل باعث میشود این داده ها حذف شوند و این اطلاعات از دست برود. در روش دوم میتوان این داده ها را پیش از آموزش مدل و با مشخص کردن طول ثابتی برای تمامی داده های ورودی و محدود کردن آنها به تعداد کلمات مشخصی کنترل کرد که من در این پروژه از روش دوم استفاده کرده ام که در ادامه و در قطعه کدهای بعدی ان را بررسی خواهیم کرد. حال میتوان نمودار همبستگی را با استفاده از کد تصویر زیر به نمایش بگذاریم.

```

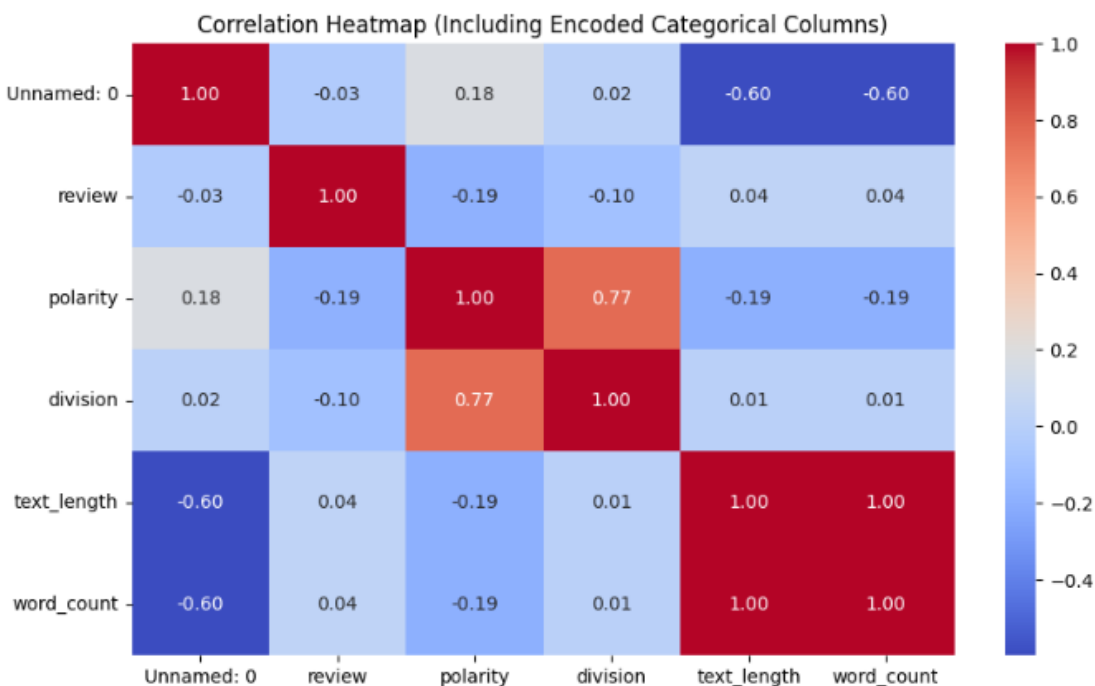
df_encoded = prepro.copy()
# to convert categorical columns to numerical
for col in df_encoded.columns:
    if df_encoded[col].dtype == 'object':
        df_encoded[col] = df_encoded[col].astype('category').cat.codes

# to show the correlation heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(df_encoded.corr(numeric_only=True), annot=True, cmap='coolwarm', fmt=".2f")
plt.title("Correlation Heatmap (Including Encoded Categorical Columns)")
plt.show()

```

تصویر ۳-۳۶ که مربوط به نمودار همبستگی دیتاست دوم

همانگونه که پیش تر به ان اشاره کردیم، این نمودار همبستگی، روابط بین ویژگی های مختلف این دیتاست را نشان می دهد. ضریب همبستگی ویژگی review با سایر ویژگی ها بسیار پایین که نشان دهنده عدم وجود همبستگی بین این ویژگی و نظرات کاربران، طول متن یا تعداد کلمات است. در مقابل اما text_length و word_count همبستگی کامل و مستقیم با یکدیگر دارند، چرا که ضریب همبستگی 1 است که منطقی است که این دو متغیر به شدت به هم وابسته باشند. همچنین، division با text_length و word_count همبستگی ضعیف مستقیم دارد چرا که ضریب همبستگی آنها 0.19 است، که نشان دهنده ارتباط جزئی بین کلاس هدف و طول متن و حتی تعداد کلمات متن است. همچنین، همبستگی نسبتاً بالایی با ضریب همبستگی 0.77 بین "polarity" و "division" وجود دارد. ویژگی "review" همبستگی ضعیفی با سایر ویژگی ها دارد که نشان دهنده عدم وجود ارتباط معنادار با دیگر فاکتورها و ویژگی ها است. به طور کلی، میتوان از این نمودار به این نتیجه قطعی رسید که نظرات کاربران و کامنت ها تقریباً مستقل از طول متن و تعداد کلمات هستند، در حالی که طول متن و تعداد کلمات به طور کامل به هم مرتبط هستند اگرچه که الگوهایی در متن ها موجودند که نشان دهنده ارتباط بین کامنت و کلاس هدف هستند که پیچیده هستند و توسط مدل در روند آموزش یاد گرفته میشوند و به کمک آنها مدل آموزش دیده و سپس کلاس بندی میکند.



تصویر ۳-۳۷ نمودار همبستگی مربوط به دیتاست دوم

۳-۳ پیش پردازش

حال که دیتاست ها را بررسی کردیم، ابتدا مشکل عدم متعادل بودن دیتاست دوم را حل کرده و پس از آن به آماده سازی داده ها برای آموزش و تست مدل و سپس مدل را آموزش داده و تست میکنیم که در ادامه به نمایش کد آنها و شرح آنها میپردازم.

```
# to handel class imbalance problem of the second dataset
X = prepro['review']
y = prepro['division']
df = pd.DataFrame({'review': X, 'division': y})

# to balance it by oversampling
max_count = df['division'].value_counts().max()
df_balanced = pd.concat([
    group.sample(max_count, replace=True, random_state=42)
    for _, group in df.groupby('division')
])
df_balanced = df_balanced.sample(frac=1, random_state=42).reset_index(drop=True)
```

تصویر ۳-۳۸ کد مربوط به متعادل سازی دیتاست دوم

این کد برای مقابله با مشکل عدم تعادل کلاس ها (class imbalance) در دیتاست دوم میپردازد که به طور کلی مشکل عدم تعادل کلاس ها زمانی رخ می دهد که تعداد نمونه ها در کلاس های مختلف به صورت نابرابر است؛ برای مثال اگر داده های منفی زیاد ولی داده های مثبت کم باشند، مدل به درستی کلاس های کم نمونه را

یاد نمی‌گیرد. در این قطعه کد ابتدا با تعریف متغیرهای X و Y، ویژگی‌های review و division از دیتاست دوم استخراج میکنیم، که به ترتیب شامل متن کامنت‌ها و کلاس‌های احساسی آنها هستند. سپس این دو ستون در قالب یک دیتافریم جدید به نام df ترکیب می‌شوند. برای حل این مشکل، با استفاده از تکنیک Oversampling، ابتدا بیشترین تعداد نمونه در بین کلاس‌ها را با استفاده از دستور value_counts().max() محاسبه میکنیم و سپس برای هر کلاس، رکورد‌ها را با جایگزینی (replace=True) تا رسیدن به آن مقدار نمونه برداری میکنیم. در نهایت، داده‌های ترکیب‌شده با استفاده از دستور pd.concat() به صورت تصادفی ترکیب میشوند و با دستور sample(frac=1) مخلوط میشوند و با دستور reset_index(drop=True) اندیس‌دهی مجدد می‌شوند. این کار منجر میشود که یک دیتافریم متعادل‌شده (df_balanced) داشته باشیم.

```
# to rename columns
df_balanced = df_balanced.rename(columns={'review': 'text', 'division': 'sentiment'})

# to split the second dataset
prepro_train, prepro_test = train_test_split(df_balanced, test_size=0.1, random_state=42)

# to combine datasets
combined_train_df = pd.concat([train_df, prepro_train], ignore_index=True)
combined_test_df = pd.concat([test_df, prepro_test], ignore_index=True)

combined_train_df = combined_train_df.sample(frac=1, random_state=42).reset_index(drop=True)
combined_test_df = combined_test_df.sample(frac=1, random_state=42).reset_index(drop=True)

# to remove irrelevant columns
columns_to_keep = ['text', 'sentiment']
combined_train_df = combined_train_df[columns_to_keep].dropna()
combined_test_df = combined_test_df[columns_to_keep].dropna()
```

تصویر ۳-۳۹ کد مربوط به ترکیب کردن دیتاست‌ها و حذف ویژگی‌های نامرتب

با استفاده از این کد چند مرحله‌ی مهم را برای آماده‌سازی و ترکیب دو دیتاست را انجام می‌دهیم به این صورت که ابتدا با استفاده از دستور rename، ستون‌های دیتافریم متعادل‌شده را از review به text و از division به sentiment تغییر نام میدهیم چرا که ستون‌های کامنت و کلاس احساسی دیتاست دیگر این نام‌ها را دارند و برای ترکیب کردن این دو دیتاست باید ستون‌ها را همان‌نام کنیم تا ساختار هماهنگی با هم داشته باشند. سپس دیتاست دوم با استفاده از دستور train_test_split به دو بخش آموزش و تست تقسیم میکنیم به این صورت که 90 درصد آن را به آموزش و 10 درصد باقی را به تست اختصاص میدهیم. این نسبت درصدی را انتخاب کرده‌ام چرا که برای دیتاست‌های کوچک بهتر است درصد بالاتر را به آموزش اختصاص دهیم تا مدل تعداد نمونه کافی ببیند و امکان آموزش دیدن از الگوهای آنها را داشته باشد. در ادامه، با استفاده از دستور pd.concat، بخش آموزش دیتاست دوم را به دیتاست آموزشی دیتاست اول و بخش تست آن را به دیتاست تست دیتاست اول اضافه میکنیم تا مجموعه داده‌های بزرگ‌تری برای آموزش و ارزیابی فراهم و

ایجاد کنیم. برای جلوگیری از هرگونه ترتیبی در داده‌ها که می‌تواند مدل را متأثر کند، این دو دیتافریم ترکیبی را با استفاده از دستور `sample(frac=1)` به صورت تصادفی درهم‌ریخته و ترکیب میکنیم و با استفاده از دستور `reset_index(drop=True)` اندیس‌دهی مجدد میکنیم. در پایان، تنها ستون‌ها و ویژگی‌های مرتبط با هدف پروژه را نگه می‌داریم و باقی را حذف میکنیم، یعنی درواقع ویژگی‌های `text` و `sentiment` را نگه داشته ایم و مقادیر گم‌شده‌ی این ستون‌ها را (NaN) با استفاده از دستور `dropna()` حذف میکنیم.

۳-۴ آموزش مدل BILSTM

```
# to tokenize and encode
max_words = 10000
max_len = 100

tokenizer = Tokenizer(num_words=max_words, oov_token="<OOV>")
tokenizer.fit_on_texts(combined_train_df['text'].astype(str))
sequences = tokenizer.texts_to_sequences(combined_train_df['text'].astype(str))
X = pad_sequences(sequences, maxlen=max_len, padding='post')

label_encoder = LabelEncoder()
y = label_encoder.fit_transform(combined_train_df['sentiment'])
```

تصویر ۳-۴۰ کد مربوط به توکنایزر

با استفاده از این کد مرحله‌ی تبدیل متن‌ها به قالب عددی را انجام میدهیم چرا که قالب ورودی‌ها برای مدل یادگیری عمیق باید به این صورت باشد برای آنکه بتوانیم ادامه کد را درک کنیم باید ابتدا بدانیم که قالب این ورودی‌ها به چه صورت است و چگونه آنها تولید میشوند به همین جهت ابتدا به توضیح این بخش میپردازیم و بعد ادامه کد را شرح میدهیم.

قالب ورودی مدل‌های یادگیری عمیق به صورت یک ماتریس دوبعدی است که یک ستون و n ردیف دارد. هر ردیف یک توالی عددی (sequence) است که هر کدام از این توالی‌ها بدست آمده از هر کامنت دیتاست آموزش هستند. (هر توالی عددی دارای ساختار نمونه‌ای $[2,4,5,8,...,0]$ هستند) هر یک از این اعداد این توالی‌ها نماینده یک کلمه از کامنت مربوطه هستند و این اعداد در واقع اندیس (index) مربوط به آن کلمه در دایره لغات توکنایزر هستند یعنی در توکنایزر که دارای m کلمه‌ی پرتکرار داده آموزش میباشد، را داریم که هریک اندیس دارند هرچه یک کلمه پرتکرارتر باشد اندیس آن کمتر است و اندیس اول متعلق به عبارت `<OOV>` میباشد که به معنای خارج از دایره لغات (out of vocabulary) است. حال برای درک بهتر باید بدانیم که توکنایزر (tokenizer) چیست و چه کاری را انجام میدهد. یک توکنایزر متن خام کامنت

ها را به توالی‌هایی از اعداد تبدیل می‌کند، که در آن به هر کلمه منحصر به فرد یک اندیس عدد صحیح منحصر به فرد اختصاص داده می‌شود این کار باعث می‌شود متن توسط شبکه‌های عصبی قابل پردازش باشد پس درواقع چیزی که این ماتریس و توالی‌های عددی که راجع به آن صحبت کردیم، توسط توکنایزر ساخته میشوند. پس روند کلی به این صورت است که در ابتدای کد ما دو متغیر را تعیین میکنیم اول طول داده که حداکثر تعداد واژه‌های مورد استفاده (max_words)، که تعداد کلمات دایره لغات توکنایزر را و در این پروژه ان را 10000 در نظر گرفته‌ام، مشخص میکند و طول هر توالی (max_len)، که طول توالی‌ها را مشخص میکند و در این پروژه ان را 100 در نظر گرفته‌ام، هستند. سپس همانطور که اشاره کردیم توکنایزر 1000 کلمه پرتکرار را پیدا میکند به پرتکرارترین کمترین اندیس را میدهد و انها را در حافظه نگه میدارد، سپس تک تک کامنت‌های موجود در دیتاست آموزش را تبدیل به توالی عددی میکند و با رسیدن به هر کلمه به ان اندیس مربوط به همان کلمه را بر اساس این دایره لغات میدهد و اگر به کلمه ای برسد که در این دایره لغات نباشد به ان اندیس صفر را میدهد که متعلق به <OOV> است. پس از توکنایز، عمل اندازه کردن متن‌ها (padding) را انجام میدهم که متغیر طول توالی‌ها را برای ان تعیین کرده ایم. خروجی توکنایزر را به عنوان ورودی برای اندازه کردن استفاده میکنیم که به این صورت عمل میکند که این توالی‌ها را به طول مشخصی میرسانند یعنی اگر توالی طولانی تر باشد ان را کات میکند تا به 100 کلمه برسد و اگر کوتا‌تر باشد به انتهای ان صفر اضافه میکند تا به طول 100 کلمه برسد. پس از اعمال این دو مرحله داده آماده استفاده به عنوان ورودی برای شبکه عصبی میشود. پس متن‌ها به لیست‌هایی از اعداد تبدیل می‌شوند که هر عدد نماینده‌ی یک کلمه است که با استفاده از texts_to_sequences و سپس با استفاده از pad_sequences همگی به طول یکسان max_len درمی‌آیند. در نهایت، ستون و ویژگی sentiment که برچسب‌های کلاس‌ها هستند، با استفاده از LabelEncoder به مقادیر عددی (۰، ۱، ۲) تبدیل می‌شوند و در y ذخیره می‌شوند.

```
# to set class weights
class_weights = compute_class_weight(class_weight='balanced', classes=np.unique(y), y=y)
class_weights_dict = dict(enumerate(class_weights))

# to split train and validation sets
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)
```

تصویر ۳-۴۱ کد تنظیم وزن کلاس‌ها و تقسیم دیتاست به دو بخش آموزش و تست برای هر دوره آموزش در این قطعه کد ابتدا برای مقابله با مشکل عدم تعادل کلاس‌ها (class imbalance) در داده‌های آموزشی، وزن‌های کلاس را محاسبه می‌کند. با استفاده از تابع compute_class_weight و

پارامتر `class_weight='balanced'`، وزن‌هایی برای هر کلاس محاسبه می‌شود که با توجه به فراوانی آن کلاس در کل داده‌ها تعیین می‌شود؛ کلاس‌هایی که نمونه‌های کمتری دارند وزن بیشتری به آنها اختصاص داده می‌شود. این وزن‌ها به صورت دیکشنری (`class_weights_dict`) ذخیره می‌شوند تا هنگام آموزش مدل استفاده شوند.

سپس با استفاده از دستور `train_test_split`، داده‌ها به دو بخش آموزش (`x-train, y-train`) و تست (`x-test, y-test`)، با نسبت 80 به 20 تقسیم می‌شوند که 80 درصد آن برای بخش آموزش و 20 درصد آن برای تست می‌باشد این دو بخش از دیتاست آموزش گرفته شده و برای هر دوره آموزش (`epoch`) از آنها استفاده خواهیم کرد.

```
# to load glove
embedding_dim = 100
embedding_index = {}
glove_path = '/kaggle/input/glove-global-vectors-for-word-representation/glove.6B.100d.txt'
with open(glove_path, encoding='utf8') as f:
    for line in f:
        values = line.split()
        word = values[0]
        coefs = np.asarray(values[1:], dtype='float32')
        embedding_index[word] = coefs
```

تصویر ۳-۴۲ کد بارگذاری بردارهای آموزش دیده GloVe

در این قطعه کد مجموعه GloVe (global vectors for word representation) را بارگذاری کردیم که به صورت کلی GloVe یک مجموعه آموزش دیده است که آن را میتوان تعبیه کرد که این مجموعه دارای یک دیکشنری از لغات و بردارهایش میباشد یعنی یک ساختار دیکشنری دارد که در آن هر کلمه یک بردار دارد که این اگر دو کلمه در حالت معمول در کنار هم استفاده شوند یا هر دو به یک موضوع مرتبط باشند انگاه اعداد بردارهای آنها نیز به هم نزدیک خواهد بود تا از این طریق مدل معنا و ارتباط آنها را بهتر درک کند. اعداد این بردارها بر اساس کاربرد آن کلمه در پیکره های متنی بزرگ مانند ویکی پدیا تعیین شده است. این مجموعه را در سه حالت 60، 100 و 200 میتوان استفاده کرد که این اعداد ابعاد این بردارها هستند یعنی در مدل GloVe 60 این کلمات به بردارهایی 60 بعدی تبدیل شده اند و در حالت 100 و 200 به بردارهایی با 100 و 200 بعد تبدیل شده اند. حال که با GloVe آشنا شدیم، به شرح این کد میپردازیم. در این قطعه کد ابتدا متغیر `embedding_dim` را روی 100 تنظیم کرده ایم که به این معناست که از نسخه ای از GloVe استفاده می‌شود که هر کلمه را با یک بردار 100 بعدی نمایش می‌دهد. سپس دیکشنری `embedding_index` را برای نگهداری نگاشت بین کلمات و بردارهایشان تعریف می‌کنیم سپس

دایرکتوری glove.6B.100d.txt که در محیط Kaggle به عنوان یکی از دیتاست های ورودی تعریف کرده ایم را می‌دهیم، تا به عنوان منبع داده برداری در نظر گرفته شود. سپس با استفاده از حلقه for هر خط از فایل که شامل یک کلمه و بردار عددی متناظر آن است، خوانده می‌شود. کلمه به عنوان کلید و بردار عددی آن به صورت آرایه در embedding_index ذخیره می‌شود. در نهایت، این دیکشنری شامل تمام بردارهای کلمات موجود در GloVe است که برای ساخت ماتریس embedding در مدل استفاده خواهد شد.

```
embedding_matrix = np.zeros((max_words, embedding_dim))
for word, i in tokenizer.word_index.items():
    if i < max_words:
        embedding_vector = embedding_index.get(word)
        if embedding_vector is not None:
            embedding_matrix[i] = embedding_vector
```

تصویر ۳-۴۳ کد مربوط به ماتریس embedding مدل

از این قطعه کد برای ساخت ماتریس embedding استفاده کرده ایم که در مدل یادگیری عمیق به عنوان وزن های لایه ی embedding استفاده خواهد شد. ابتدا یک ماتریس با مقدار اولیه صفر با اندازه (max_words, embedding_dim) ایجاد می‌کنیم، که max_words حداکثر تعداد واژه‌هایی است که در قطعه کد قبل آنها را 1000 تعیین کردیم، و embedding_dim تعداد ابعاد بردارهای کلمات که در این پروژه ان را 100 تعیین کرده ام. سپس با استفاده از یک حلقه تمام واژه نامه و دایره لغات تولید شده توسط tokenizer، را طی می‌کنیم. اگر ایندکس واژه که نام دارد، کمتر از max_words باشد، چک می‌شود که آیا بردار متناظر آن واژه در دیکشنری embedding_index که از GloVe بارگذاری شده است، وجود داشت، ان بردار را در همان اندیس به ماتریس اضافه میکند اما اگر وجود نداشت، هیچ مقداری به ماتریس در ان اندیس اضافه نمیشود و مقدار ان اندیس صفر باقی می‌ماند. نتیجه ی نهایی یک ماتریس آماده برای مقداردهی اولیه به لایه ی embedding در مدل است که در آن برای هر واژه ای که در GloVe دیده شده، از بردارهای از پیش آموزش دیده استفاده می‌شود، و برای بقیه واژه ها بردار صفر باقی می‌ماند. پس هر ردیف این ماتریس حاوی یک بردار است و اندیس این بردار برابر است با اندیس کلمه متناظر در ماتریس tokenizer است.

```
# to build and train BiLSTM model
model = Sequential([
    Embedding(max_words, embedding_dim, weights=[embedding_matrix], input_length=max_len, trainable=True),
    Dropout(0.2),
    Bidirectional(LSTM(128)),
    Dropout(0.2),
    Dense(64, activation='relu', kernel_regularizer=regularizers.l2(0.001)),
    Dropout(0.2),
    Dense(len(np.unique(y)), activation='softmax')
])
```

تصویر ۳-۴۴ کد مربوط به مدل BiLSTM

این کد مربوط به مدل BiLSTM است که در آن لایه ها و پارامترهایش را مشخص کرده ایم.

لایه ی embedding از ماتریس embedding از پیش ساخته شده (که با GloVe مقاداردهی اولیه شده) استفاده می کند از انجایی که پارامتر trainable را true قرار داده ایم این مقادارهای اولیه بر اساس داده های دیتاست آموزش و هدف ما از پروژه تغییر خواهند کرد و مدل آنها را تنظیم خواهد کرد اما این مقاداردهی اولیه وزن های اولیه از این جهت دارای اهمیت است که مدل از حالتی بهتر و پیشرفته تر از حالت صفر شروع میکند به طور کلی این لایه embedding مسئول تبدیل توکن های متنی به بردارهای عددی است که برای مدل دارای اهمیت هستند یعنی بعد از این لایه به عنوان خروجی تمامی کامنت های داده آموزش را خواهیم داشت که به طول 100 کلمه محدود شده اند و هر کلمه دارای 100 بعد است و همانطور که به آن اشاره کردیم، پارامتر trainable=True باعث می شود وزن های این لایه در طول آموزش قابل به روزرسانی باشند.

لایه ی Dropout (0.2) برای جلوگیری از بیش برآزش (overfitting) است و در هر مرحله آموزش، به طور تصادفی 20 درصد از ورودی های لایه ی بعدی را روی 0 تنظیم می کند..

لایه ی Bidirectional LSTM یک لایه LSTM با 128 نورون که به صورت دوطرفه اجرا می شود، یعنی ورودی را هم از ابتدا به انتها و هم از انتها به ابتدا پردازش می کند. این باعث می شود که مدل زمینه ی قبل و بعد هر کلمه را در نظر بگیرد و آن را بهتر درک کند. لایه ی Dropout (0.2) دومین لایه Dropout همانند لایه Dropout، برای کاهش احتمال بیش برآزش است و در هر مرحله آموزش، به طور تصادفی 20 درصد از ورودی های لایه ی بعدی را روی 0 تنظیم می کند. لایه ی Dense یک لایه ی تماماً متصل (fully connected) است که دارای 64 نورون و تابع فعال سازی ReLU و یک regularization نوع L2 با مقدار 0.0001 اسن که کمک می کند از بیش برآزش جلوگیری شود. خروجی BiLSTM با ابعاد بالا را به یک

نمایش فشرده‌تر و قابل یادگیری‌تر تبدیل می‌کند و relu به آموزش دیدن مدل به صورت غیرخطی کمک می‌کند و L2 Regularization، با کوچک نگه داشتن وزن‌ها، احتمال بیش‌برازش را کاهش می‌دهد.

سومین لایه Dropout برای کنترل پیچیدگی مدل است و به طور تصادفی 20 درصد از ورودی‌های لایه ی بعدی را روی 0 تنظیم می‌کند.. لایه ی نهایی Dense که لایه خروجی است که به تعداد کلاس‌ها نورون دارد (که برای این پروژه 3 عدد است که منفی، خنثی و مثبت هستند) و از تابع softmax برای پیش‌بینی احتمالات استفاده می‌کند یعنی توزیع احتمال را روی کلاس‌ها ارائه می‌دهد.

به صورت کلی شکل خروجی و ورودی هر لایه به این صورت است که در جدول زیر مشاهده میکنید.

لایه‌های مدل	شکل ورودی	شکل خروجی	کاری که این لایه مدل میکند
Input (tokenized & padded)	(35758, 100)	(35758, 100)	هر یک از کامنت‌های داده آموزش که 35758 عدد هستند به صورت دنباله‌ای از 100 اندیس نمایش داده می‌شود که هر اندیس نشان دهنده میزان پرتکرار بودن آن است به این صورت که هرچه پرتکرارتر، عدد اندیس کمتر است.
Embedding	(35758, 100)	(35758, 100, 100)	با استفاده از هر اندیس، کلمه مربوطه را یافته و کلمه را با استفاده از GloVe به یک بردار 100 بعدی تبدیل می‌کند. بنابراین هر کامنت به یک ماتریس با شکل (100 100,) تبدیل می‌شود که نشان دهنده آن است که هر کامنت با طول 100 کلمه میباشد و هر کلمه به یک بردار 100 بعدی تبدیل میشود.
Dropout (0.2)	(35758, 100, 100)	(35758, 100, 100)	به طور تصادفی 20 درصد از بردارها را در طول آموزش صفر قرار می‌دهد تا از بیش‌برازش جلوگیری شود.
Bidirectional LSTM (128 units)	(35758, 100, 100)	(35758, 256)	توالی‌های 100 کلمه‌ای را در هر دو جهت پردازش می‌کند. هر جهت 128 واحد خروجی می‌دهد که در مجموع 256 واحد برای هر توالی میشود.
Dropout (0.2)	(35758, 256)	(35758, 256)	به طور تصادفی 20 درصد از خروجی لایه ی قبل را حذف میکند.
Dense (64, relu)	(35758, 256)	(35758, 64)	لایه کاملاً متصل که ابعاد را به ۶۴ کاهش می‌دهد و غیرخطی بودن را اضافه می‌کند.
Dropout (0.2)	(35758, 64)	(35758, 64)	باز هم، 20 درصد از خروجی‌ها را جهت بهبود تعمیم پذیری مدل حذف می‌کند.
Dense (3 classes, softmax)	(35758, 64)	(35758, 3)	لایه پیش‌بینی نهایی که احتمال هر یک از ۳ کلاس احساسی را ارائه می‌دهد.

تصویر ۳-۴۵ شرح عملکرد لایه های مدل BILSTM

حال به شرح ادامه کد می‌پردازیم که در تصویر زیر مشاهده میکنید.

```
model.compile(loss='sparse_categorical_crossentropy',
              optimizer=tf.keras.optimizers.Adam(learning_rate=0.0005),
              metrics=['accuracy'])

early_stop = EarlyStopping(monitor='val_loss', patience=2, restore_best_weights=True)
lr_schedule = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=2, verbose=1)
```

تصویر ۳-۴۶ کد مربوط به کامپایل مدل و تنظیم callbacks

این بخش از کد مربوط به کامپایل مدل و تنظیم callback ها میباشد و نقش مهمی در پایدار سازی و بهینه سازی فرآیند آموزش دارد. در ابتدا، مدل با استفاده از دستور model.compile(...) آماده‌ی آموزش می‌شود. تابع زیان (loss) از نوع sparse_categorical_crossentropy انتخاب شده چون برچسب‌ها به صورت عددی هستند همچنین از بهینه‌ساز Adam با نرخ یادگیری 0.0005 استفاده کرده ایم تا فرآیند یادگیری با گام‌های کوچک‌تر و پایداری بیشتری انجام شود. معیار ارزیابی مدل نیز دقت (accuracy) در نظر گرفته شده که برای تحلیل عملکرد در طول آموزش و اعتبارسنجی است اگرچه که در انتها مدل را با معیارهای بیشتری نیز می‌سنجیم. در ادامه، دو callback برای کنترل بهتر آموزش در نظر گرفته شده‌اند EarlyStopping. فرآیند آموزش را زیر نظر دارد و اگر به مدت دو دوره متوالی عملکرد مدل بر اساس loss متعلق به اعتبارسنجی (val_loss) بهبود نیابد، مدل را متوقف می‌کند و وزن‌هایی را که بهترین عملکرد را داشته‌اند بازمی‌گرداند. از طرفی دیگر، ReduceLROnPlateau نیز در صورت عدم بهبود عملکرد مدل بر اساس loss متعلق به اعتبارسنجی (val_loss) به مدت دو دوره، نرخ یادگیری را به نصف کاهش می‌دهد تا مدل بهتر در نواحی بهینه تنظیم شود. این دو کمک می‌کنند تا از overfitting جلوگیری شود، نرخ یادگیری به صورت پویا تنظیم گردد و مدل با بهترین وزن‌ها و عملکرد آموزش ببیند که در واقع به زبانی ساده نرخ یادگیری تعیین میکند مدل در هر مرحله چقدر وزن‌ها را تغییر دهد تا به جواب بهتر برسد. اگر این عدد خیلی بزرگ باشد، مدل ممکن است از جواب درست رد شود و یاد نگیرد. اگر خیلی کوچک باشد، مدل خیلی کند یاد می‌گیرد یا اصلاً به جواب خوب نمی‌رسد. پس باید عددی مناسب انتخاب شود تا مدل خوب و درست آموزش ببیند. ادامه کد را در تصویر زیر مشاهده میکنید.

```
history = model.fit(X_train, y_train,
                   validation_data=(X_val, y_val),
                   epochs=10,
                   batch_size=64,
                   callbacks=[early_stop, lr_schedule],
                   class_weight=class_weights_dict)
```

تصویر ۳-۴۷ کد مربوط به فرایند آموزش مدل

با کمک این قطعه کد مدل را آموزش داده ایم و پارامترهایی مانند داده‌های آموزش (X_{train}, y_{train}) و داده‌های اعتبارسنجی (X_{val}, y_{val}) را مشخص کرده ایم. مدل در هر بار تکرار (epoch) کل داده‌های آموزش را بررسی می‌کند و بعد از هر epoch روی داده‌های اعتبارسنجی آزمایش می‌شود. تعداد epoch ها روی 10 تنظیم شده و هر بار 64 نمونه با هم پردازش می‌شوند که batch size نیز با استفاده از class_weight، به کلاس‌هایی که نمونه‌های کمتری دارند وزن بیشتری داده می‌شود تا مدل آن‌ها را بهتر یاد بگیرد.

علاوه بر این، دو callback برای آموزش بهتر اضافه شده‌اند که یکی از آنها EarlyStopping است که اگر مدل دیگر بهتر نشود، زودتر آموزش را متوقف می‌کند، و دیگر ReduceLROnPlateau که اگر بهبودی در val_loss رخ ندهد، نرخ یادگیری را کاهش می‌دهد. در نهایت نتیجه و روند آموزش نیز در متغیر history ذخیره می‌شود تا بعداً بتوانیم آن را به کمک نمودارها و معیارهای دیگر تحلیل کنیم البته لازم به ذکر است که val_loss یا Validation Loss نشان می‌دهد که مدل هنگام آزمایش روی داده‌های اعتبارسنجی (validation data) چقدر خطا دارد. به‌طور ساده، در هر مرحله از آموزش، مدل پس از یادگیری از داده‌های آموزش، روی یک دسته‌ی جداگانه از داده‌ها که داده‌های validation است که قبلاً ندیده، آزمایش می‌شود. نتیجه این آزمایش یک عدد به نام val_loss است که مقدار اشتباه مدل روی داده‌های اعتبارسنجی را نشان می‌دهد. اگر val_loss کاهش پیدا کند، یعنی مدل روی داده‌های جدید بهتر عمل می‌کند. اما اگر زیاد شود، ممکن است مدل دچار overfitting شده باشد؛ یعنی فقط داده‌های آموزش را خوب یاد گرفته باشد و روی داده‌های ناآشنا عملکرد خوبی نداشته باشد. در قطعه کد عکس زیر ادامه کد را مشاهده مکنید که در ادامه آن را شرح داده‌ام.

۴. فصل چهارم

نتایج

حال که مدل را آموزش داده ایم در این بخش به شرح و توضیح قطعه کد های مربوط به تست مدل، مصورسازی عملکرد مدل و بررسی و تحلیل آنها میپردازیم.

```
# to test model
test_sequences = tokenizer.texts_to_sequences(combined_test_df['text'].astype(str))
X_test = pad_sequences(test_sequences, maxlen=max_len, padding='post')

pred_probs = model.predict(X_test)
pred_classes = np.argmax(pred_probs, axis=1)

# to evaluate model
if 'sentiment' in combined_test_df.columns:
    test_eval_df = combined_test_df.dropna(subset=['sentiment']).copy().reset_index(drop=True)
    true_labels = label_encoder.transform(test_eval_df['sentiment'])
    pred_eval_labels = pred_classes[:len(test_eval_df)]

    test_accuracy = np.mean(pred_eval_labels == true_labels)
    print(f"\nTest Accuracy: {test_accuracy:.4f}")

    print("\nClassification Report:")
    print(classification_report(true_labels, pred_eval_labels, target_names=label_encoder.classes_))

    print("Confusion Matrix:")
    print(confusion_matrix(true_labels, pred_eval_labels))

print(f"Loaded {len(embedding_index)} GloVe vectors")
print(f"Used {np.count_nonzero(np.count_nonzero(embedding_matrix, axis=1))} in embedding matrix")
```

تصویر ۳-۴۸ کد مربوط به تست مدل

این قطعه کد برای آزمون و ارزیابی مدل آموزش دیده میباشد که در آن ابتدا متن های موجود در داده ی تست با استفاده از همان توکنایزر (Tokenizer) تبدیل به توالی های عددی می کنیم از انجایی که این توالی ها ممکن است طول های مختلفی داشته باشند، با استفاده از تابع pad_sequences آن ها به طولی ثابت 100 کلمه تنظیم می کنیم در واقع با این فرآیند تضمین می کنیم که داده ی تست همان ساختار داده ی آموزش را دارد.

پس از آماده سازی داده ها، مدل آموزش دیده برای پیش بینی احساسات و کلاس بندی کامنت های دیتاست تست استفاده میکنیم. مدل ابتدا برای هر نمونه احتمال تعلق آن به هر کلاس را برمی گرداند (pred_probs) که خروجی لایه softmax است. سپس با استفاده از تابع argmax، کلاسی که بیشترین احتمال را دارد را برای هر نمونه انتخاب میکنیم که به عنوان پیش بینی نهایی (pred_classes) در نظر گرفته می شود. همچنین اگر داده های تست شامل برچسب های واقعی و اصلی یعنی همان ستون sentiment باشند، پیش بینی مدل با این برچسب ها مقایسه می شود البته لازم به ذکر است که ابتدا نمونه هایی که برچسب ندارند حذف می شوند و سپس برچسب های متنی به عدد تبدیل می گردند تا بتوان آن ها را با پیش بینی ها مقایسه کرد. سپس، دقت مدل محاسبه شده و گزارشی کامل از عملکرد کلی مدل که شامل معیار های precision، recall و F1-score میباشد، برای هر کلاس نمایش داده و چاپ میشود. همچنین، یک ماتریس سردرگمی نشان داده می شود تا

توزیع پیش‌بینی‌ها را دقیق‌تر بررسی کنیم. در پایان نیز تعداد کل بردارهای GloVe بارگذاری شده و تعداد آن‌هایی که در مدل واقعاً استفاده شده‌اند، چاپ می‌شود.

نتایج این کد را در چند عکس بعد خواهید دید که آنها را به ترتیب تحلیل خواهیم کرد.

```
447/447 ----- 125s 263ms/step - accuracy: 0.5299 - loss: 1.0165 - val_accuracy: 0.6952 - val_loss: 0.7341 - learning_rate: 5.0000e-04
Epoch 2/10
447/447 ----- 117s 261ms/step - accuracy: 0.7146 - loss: 0.7042 - val_accuracy: 0.7545 - val_loss: 0.6270 - learning_rate: 5.0000e-04
Epoch 3/10
447/447 ----- 112s 251ms/step - accuracy: 0.7602 - loss: 0.6076 - val_accuracy: 0.7700 - val_loss: 0.5838 - learning_rate: 5.0000e-04
Epoch 4/10
447/447 ----- 118s 263ms/step - accuracy: 0.7875 - loss: 0.5388 - val_accuracy: 0.7838 - val_loss: 0.5565 - learning_rate: 5.0000e-04
Epoch 5/10
447/447 ----- 113s 254ms/step - accuracy: 0.8019 - loss: 0.5026 - val_accuracy: 0.7837 - val_loss: 0.5516 - learning_rate: 5.0000e-04
Epoch 6/10
447/447 ----- 118s 264ms/step - accuracy: 0.8160 - loss: 0.4711 - val_accuracy: 0.7935 - val_loss: 0.5437 - learning_rate: 5.0000e-04
Epoch 7/10
447/447 ----- 113s 253ms/step - accuracy: 0.8275 - loss: 0.4407 - val_accuracy: 0.7933 - val_loss: 0.5493 - learning_rate: 5.0000e-04
Epoch 8/10
447/447 ----- 0s 234ms/step - accuracy: 0.8390 - loss: 0.4114
Epoch 8: ReduceLROnPlateau reducing learning rate to 0.000250000118743628.
447/447 ----- 113s 253ms/step - accuracy: 0.8390 - loss: 0.4114 - val_accuracy: 0.7914 - val_loss: 0.5444 - learning_rate: 5.0000e-04
140/140 ----- 8s 55ms/step

Test Accuracy: 0.7822
```

تصویر ۳-۴۹ بخش اول نتایج مدل

این تصویر همانطور که له ان اشاره کردم مربوط به روند آموزش و ارزیابی مدل BILSTM ای است که در این پروژه آموزش داده ام که ابتدا، دقت (accuracy) مدل روی داده‌های آموزش از مقدار حدود 53 درصد شروع شده و در طول 8 دوره‌ی آموزش به حدود 84 درصد رسیده است که نشان‌دهنده‌ی بهبود قابل توجهی در یادگیری مدل است. همچنین کاهش پیوسته در مقدار خطا (loss) در داده‌های آموزش، از حدود 1.01 به 0.41، را مشاهده می‌کنیم که در واقع بیانگر این است که مدل به تدریج بهتر شده و خطاهایش کمتر شده است همچنین در بخش اعتبارسنجی (validation) نیز دقت مدل از حدود 69 درصد شروع و تا 79 درصد افزایش یافته است که نشان‌دهنده‌ی عملکرد مناسب مدل روی داده‌هایی است که در مراحل و دوره‌های آموزش استفاده نشده‌اند و توسط مدل دیده نشده‌اند مقدار خطا روی داده‌های اعتبارسنجی نیز کاهش یافته و در نهایت به عددی حدود 0.54 رسیده است اگرچه که با این حال، بعد از دوره هشتم، عملکرد مدل روی داده‌های اعتبارسنجی تقریباً ثابت شده و تغییر قابل توجهی در دقت و خطا دیده نمی‌شود. به همین دلیل مکانیزم ReduceLROnPlateau فعال شده و نرخ یادگیری را به نصف کاهش داده است تا مدل بتواند با قدم‌های کوچکتر بهتر بهینه‌سازی شود. این کار معمولاً به بهبود نهایی عملکرد مدل کمک می‌کند ولی در این پروژه، تعداد دوره‌های آموزش بیشتری رخ نداده است که نشان می‌دهد مدل به بهترین حالت رسیده است

و متوقف شده است به همین دلیل در نهایت، دقت مدل روی داده‌های تست 78.22 درصد مانده است و به عنوان نتیجه نهایی چاپ شده است که نشان می‌دهد که مدل به طور کلی توانسته است یک عملکرد قابل قبول و نسبتاً پایدار داشته باشد. رخ یادگیری (learning rate) با مقدار $5.0000e-04$ که همان 0.0005 است که در هر مرحله از آموزش مشخص میکند، مدل تا چه اندازه وزن‌های خود را بر اساس خطا (loss) به‌روزرسانی کند. مقدار کم 0.0005 باعث می‌شود مدل با گام‌های کوچک و دقیق به سمت حداقل کردن خطا حرکت کند، که به پایداری بیشتر در آموزش کمک می‌کند چرا که اگر نرخ یادگیری خیلی بزرگ باشد، مدل ممکن است از حداقل بهینه عبور کند و اگر خیلی کوچک باشد، آموزش کند و غیرکارآمد می‌شود پس این عدد معمولاً با آزمون و خطا یا از طریق تنظیمات خودکار مانند ReduceLROnPlateau تعیین می‌شود.

عدد 447 که در هر دوره (epoch) مشاهده می‌شود به تعداد دسته‌ها (batch) در هر دوره (epoch) اشاره دارد. وقتی دیتاست خود را به مدل می‌دهیم، داده‌ها به بخش‌های کوچکتر (batch) تقسیم می‌شوند، ما 35758 داده‌ی آموزشی داریم که 80 درصد آنها برای آموزش و 20 درصد برای تست بعد از هر دوره آموزش استفاده می‌شود و batch_size برابر 64 است، پس $28607 / 64$ برابر با 447 خواهد بود. حال ادامه نتایج را بررسی می‌کنیم که در تصویر زیر مشاهده می‌کنید.

```
Classification Report:
              precision    recall  f1-score   support

negative     0.80      0.78      0.79      1308
neutral      0.76      0.74      0.75      1751
positive     0.80      0.84      0.82      1395

accuracy          0.78      4454
macro avg     0.78      0.79      0.78      4454
weighted avg   0.78      0.78      0.78      4454

Confusion Matrix:
[[1020  239   49]
 [ 210 1292  249]
 [   46  177 1172]]
Loaded 400000 GloVe vectors
Used 8677 in embedding matrix
```

تصویر ۳-۵۰ بخش دوم نتایج مدل

این تصویر نتایج عملکرد مدل را در داده‌های تست نشان می‌دهد که از گزارش کلاسبندی (Classification_Report) و ماتریس سردرگمی (Confusion_Matrix) تشکیل شده است. دقت کلی مدل روی داده‌های تست 78 درصد است که نشان دهنده عملکرد مناسب مدل در تشخیص احساسات

کامنت ها است و مدل توانسته است که کامنت های هر سه کلاس منفی، خنثی و مثبت را با دقت قابل قبولی پیش بینی کند. در بخش precision، مدل کامنت های کلاس های منفی و مثبت به دقت 80 درصد رسیده که نشان می دهد از بین نمونه هایی که به عنوان این دو کلاس پیش بینی شده اند، در 80 درصد مواقع و موارد واقعا متعلق به همان کلاس بوده اند اگرچه که در دسته خنثی دقت کمی پایین تر و برابر با 76 درصد است. در معیار recall، که نشان دهنده درصد نمونه های درست شناسایی شده نسبت به کل نمونه های واقعی هر کلاس است، بهترین عملکرد را مدل برای کلاس مثبت داشته است که 84 درصد است یعنی مدل توانسته 84 درصد نمونه های مثبت واقعی را درست تشخیص دهد اگرچه که کلاس خنثی کمترین recall را دارد که 74 درصد است.

ماتریس سردرگمی (confusion matrix) نشان می دهد که بیشترین اشتباهات مدل در تشخیص نمونه های متعلق به کلاس خنثی است، که در واقع تعدادی از نمونه های خنثی به اشتباه به کلاس های منفی و مثبت نسبت داده شده اند که تعداد 210 و 249 نمونه هستند همچنین نمونه هایی از کلاس مثبت و منفی نیز اشتباهاً به عنوان کلاس خنثی پیش بینی شده اند که این موضوع می تواند نشان دهنده تشابه ویژگی های متنی در این کلاس ها باشد و نیاز به داده های بیشتر یا مدل های پیچیده تر برای بهبود عملکرد را نشان می دهد. در نهایت، مشاهده می کنیم که از بین 400,000 بردار GloVe بار گذاری شده، فقط 8677 کلمه در ماتریس جایگذاری مدل استفاده شده اند که به طور کلی این میزان کلمات برای یادگیری ویژگی های متنی مدل کافی است ولی می توان با گسترش دامنه کلمات، مدل را قوی تر کرد.

در گزارش طبقه بندی، macro avg میانگین ساده ی غیر وزنی از معیارهای precision، recall و f1-score است که این میانگین به صورت مساوی از هر کلاس در نظر گرفته می شود، بدون توجه به اینکه چه تعداد نمونه در هر کلاس وجود دارد یعنی ابتدا معیار مورد نظر مثلاً precision برای هر کلاس محاسبه می شود، سپس میانگین آن ها گرفته می شود پس این نوع میانگین در شرایطی مفید است که بخواهیم عملکرد مدل را به طور مساوی روی تمام کلاس ها بررسی کنیم، حتی اگر داده ها نامتوازن باشند. در مقابل، weighted avg میانگین وزنی همان معیارهاست، اما به تعداد نمونه ها یعنی support در هر کلاس وزن می دهد پس support نشان دهنده تعداد واقعی نمونه ها در هر کلاس در مجموعه ی تست است. بنابراین، در weighted avg، کلاس هایی که نمونه های بیشتری دارند، تأثیر بیشتری بر میانگین نهایی خواهند داشت پس این نوع میانگین در شرایطی مفید است که بخواهیم عملکرد کلی مدل را با در نظر گرفتن پراکندگی واقعی کلاس ها ارزیابی

کنیم. به طور خلاصه، اگر داده‌ها نامتوازن باشند، macro avg وضعیت تمام کلاس‌ها را به طور برابر نشان می‌دهد، ولی weighted avg وضعیت واقعی تری از دقت کلی مدل را ارائه می‌دهد.

```
plt.figure(figsize=(14,5))

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.title('Model Accuracy over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend(loc='upper right')
plt.grid(True)

plt.tight_layout()
plt.show()
```

تصویر ۳-۵۱ کد مربوط به ترسیم نمودارهای آموزش و تست مدل

این قطعه کد برای نمایش روند تغییرات دقت (Accuracy) و خطا (Loss) در طول دوره‌های آموزش مدل (Epochs) طراحی شده است. با این هدف که بتوانیم عملکرد مدل روی داده‌های آموزش و داده‌های اعتبارسنجی (Validation) در هر epoch را مشاهده کنیم. در بخش اول نمودار، با استفاده از دستور `plt.subplot(1, 2, 1)` و دقت مدل نمایش داده می‌شود که شامل دو خط است که خط آبی برای دقت مدل بر روی داده‌های آموزش و خط نارنجی برای دقت مدل بر روی داده‌های اعتبارسنجی است. همچنین در بخش دوم نیز با `plt.subplot(1, 2, 2)` روند تغییرات خطا را نشان می‌دهد، که به همان صورت برای آموزش و اعتبارسنجی تفکیک شده است. همچنین برای خوانایی عنوان‌ها و برچسب‌ها، باکس اطلاعات را که با نام `legend` مشخص می‌شود را اضافه کرده ایم و با `plt.tight_layout()` چیدمان بهینه تر و بهتر می‌شود پس در واقع دستور `plt.legend()` در باکس اطلاعات `legend` استفاده شده است؛ یعنی بخشی از نمودار که توضیح می‌دهد هر خط یا رنگ چه داده‌ای را نشان می‌دهد همچنین آرگومان `loc` محل قرارگیری آنها را روی نمودار مشخص می‌کند پس در خط `plt.legend(loc='upper right')`، باکس در بالای سمت راست نمودار نمایش داده می‌شود، و در خط `plt.legend(loc='lower right')`، باکس در پایین سمت راست قرار دارد. همچنین دستور `plt.tight_layout()` را برای تنظیم خودکار فاصله‌ها و جای‌گذاری اجزای نمودار استفاده کرده ایم تا وقتی چند نمودار با دستور `subplot()` در یک شکل رسم می‌شوند، عنوان‌ها، محورها یا باکس اطلاعاتشان روی هم نیفتند و فضای کافی داشته باشند به طور کلی تحلیل این نمودارها به ما کمک می‌کند تشخیص دهیم

که آیا مدل ما به درستی آموزش دیده است و یا دچار مشکلاتی مانند underfitting یا overfitting شده است یا خیر، برای مثال اگر دقت آموزش و اعتبارسنجی همزمان افزایش یابد و خطا کاهش پیدا کند، نشان از عملکرد خوب مدل است اما اگر دقت آموزش خیلی بالا برود ولی اعتبارسنجی ثابت یا افت کند، احتمالاً مدل overfitted شده است. همچنین اگر هر دو دقت پایین باقی بمانند، مدل احتمالاً underfitted شده است. حال به شرح ادامه کد میپردازیم.

```
# to analyse train and test datasets
total_train_samples = len(combined_train_df)
total_test_samples = len(combined_test_df)

# to calculate the words that tokenizer found in combined train dataset
total_words_in_vocab = len(tokenizer.word_index)

# to calculate words actually used in the model limited by max_words
actual_words_used = min(len(tokenizer.word_index), max_words)

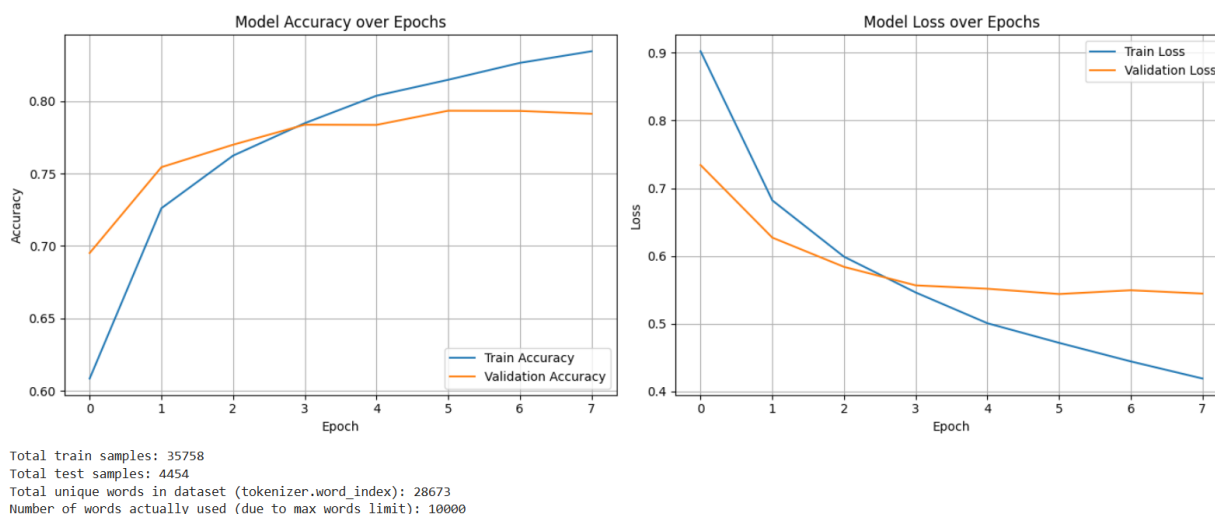
# to print the results
print("Total train samples:", total_train_samples)
print("Total test samples:", total_test_samples)
print("Total unique words in dataset (tokenizer.word_index):", total_words_in_vocab)
print("Number of words actually used (due to max_words limit):", actual_words_used)
```

تصویر ۳-۵۲ کد بررسی داده های تست و آموزش استفاده شده در فرایند آموزش مدل

این قطعه کد را برای تحلیل داده های آموزش و تست و همچنین وضعیت توکنایزر (Tokenizer) استفاده کرده ایم که در آن ابتدا با استفاده از متغیرهای total_train_samples و total_test_samples، تعداد کل نمونه های موجود در مجموعه های داده ی آموزش و تست را مشخص ذخیره میکنیم که در واقع نشان دهنده ی تعداد ردیف های موجود در هر دیتافریم ترکیبی است. سپس با متغیر total_words_in_vocab، تعداد کل کلماتی که توکنایزر روی مجموعه آموزش شناسایی کرده را محاسبه می کنیم که این مقدار برابر با طول دیکشنری tokenizer.word_index است که نشان می دهد چه تعداد واژه منحصر به فرد در داده های آموزشی وجود داشته اند. در ادامه، متغیر actual_words_used تعداد واژه هایی را مشخص می کند که واقعاً توسط مدل استفاده شده اند که این مقدار معمولاً برابر با max_words است که در تنظیمات توکنایزر تعیین کرده بودیم که برای این پروژه آن را 10000 در نظر گرفتیم که به این معناست که حتی اگر توکنایزر واژه های بیشتری شناسایی کرده باشد، فقط تا سقف تعیین شده یعنی 10000 کلمه در مدل استفاده می شود. در پایان، این مقادیر را چاپ کرده ایم تا بتوانیم در یک نگاه آنها را درک کنیم. نتیجه کد دو تصویر قبل را در تصویر زیر میتوانید مشاهده کنید که شامل دو نمودار است که عملکرد مدل در طی 8 دوره آموزش (Epochs) را نشان میدهند، سمت چپ دقت (Accuracy) مدل را نشان می دهد و مشخص است که دقت روی داده های آموزشی به صورت پیوسته افزایش یافته و از حدود 61 درصد به نزدیک 84 درصد رسیده است و همچنین

دقت داده‌های اعتبارسنجی (Validation Accuracy) نیز تا حدود دوره چهارم افزایش یافته و سپس تقریباً ثابت شده است و در نهایت در محدوده 78 درصد باقی مانده است. این روند نشان می‌دهد که مدل توانسته ساختار داده‌های آموزشی را به خوبی یاد بگیرد همچنین در نمودار سمت راست روند کاهش خطا (Loss) را برای داده‌های آموزش و اعتبارسنجی نمایش داده شده است و نشان می‌دهد که خطای آموزش به طور پیوسته کاهش یافته و این روند تا پایان آموزش حفظ شده است، در حالی که کاهش خطای اعتبارسنجی از دوره چهارم به بعد تقریباً متوقف شده است.

در پایین تصویر نیز آمار توصیفی از داده‌ها چاپ شده است که همانطور که توضیح دادیم تعداد نمونه‌های آموزش 35758، نمونه‌های تست 4454، تعداد کل واژگان یکتا موجود در کل مجموعه داده 28673 و تعداد واژگان مورد استفاده در مدل 10000 کلمه است.



تصویر ۳-۵ نمودارهای نتایج عملکرد مدل

5. فهرست مراجع و منابع

- .1 GloVe: Global Vectors for Word Representation
<https://nlp.stanford.edu/projects/glove/>
- .2 Keras: Bidirectional Layer
https://keras.io/api/layers/recurrent_layers/bidirectional/
- .3 Keras: Embedding Layer
https://keras.io/api/layers/core_layers/embedding/
- .4 Keras: EarlyStopping Callback
https://keras.io/api/callbacks/early_stopping/
- .5 Keras: ReduceLROnPlateau Callback
https://keras.io/api/callbacks/reduce_lr_on_plateau/
- .6 Scikit-learn: Classification Report
https://scikitlearn.org/stable/modules/generated/sklearn.metrics.classification_report.html
- .7 Scikit-learn: Confusion Matrix
https://scikitlearn.org/stable/modules/generated/sklearn.metrics.confusion_matrix.html